

**UNIVERSITY OF GHANA
COLLEGE OF BASIC AND APPLIED SCIENCES
DEPARTMENT OF COMPUTER SCIENCE**

**NEXZ: AN ADAPTIVE NETWORK SECURITY MONITORING AND
LEARNING ENVIRONMENT WITH AN AI-POWERED INTRUSION
DETECTION ENGINE**

*A Final-Year Computer Science Project Report Submitted to the Department of Computer Science,
University of Ghana, Legon*

**FINAL-YEAR B.SC. COMPUTER SCIENCE CAPSTONE PROJECT REPORT
DEPARTMENT OF COMPUTER SCIENCE
September 2026**

DECLARATION

I hereby declare that this project report, titled 'NEXz: An Adaptive Network Security Monitoring and Learning Environment with an AI-Powered Intrusion Detection Engine', is the result of my own original research and software implementation conducted in the Department of Computer Science, University of Ghana, Legon. Where contributions of other authors or existing software frameworks are involved, every effort has been made to indicate this clearly through citations and acknowledgements.

Student Signature

Date

CERTIFICATION

This project report has been submitted for examination with my approval as supervisor:

Supervisor Signature

Date

Department of Computer Science, University of Ghana, Legon

ABSTRACT

Context: The expansion of computer networks across educational institutions and enterprises in developing economies has increased exposure to cyber threats. Traditional Network Intrusion Detection Systems (NIDS) rely either on rigid signature databases incapable of detecting zero-day exploits or on static machine learning models that suffer severe degradation due to concept drift in non-stationary traffic streams. Furthermore, enterprise monitoring solutions are financially prohibitive and lack educational transparency for academic instruction.

Aim: This project designs, implements, and evaluates NEXz (Network Intelligence, Education and eXploration System), a unified, lightweight network security monitoring and learning environment featuring an online machine learning intrusion detection engine capable of continuous adaptation on commodity workstation hardware.

Method: An iterative software engineering approach was employed. The pipeline integrates: (1) asynchronous Layer 2/3/4 packet capture via Scapy, (2) bidirectional 5-tuple flow aggregation with 2.0-second inactivity windowing extracting 7 statistical features, (3) online Adaptive Random Forest (ARF) stream classification using an ensemble of Hoeffding Adaptive Trees, (4) statistical concept drift monitoring via ADWIN, (5) localized per-device behavioral anomaly profiling using Half-Space Trees bounded by a 200-device LRU cache, (6) centralized sliding-window alert deduplication (30s window), (7) contextual Explainable AI diagnostics, (8) a 7-protocol educational academy, (9) a multi-format external network telemetry import, replay, and benchmark evaluation subsystem (supporting PCAP, CSV, JSON, Zeek conn.log, Suricata EVE, and threat watchlists), and (10) a multi-criteria research telemetry workbench.

Results: Empirical evaluation under controlled normal and attack traffic (SYN floods, port scans, exfiltration) demonstrated 98.93% classification accuracy (F1-score: 97.28%), outperforming an offline Random Forest benchmark (94.95%). Total processing latency averaged 14.6 milliseconds per flow, memory usage remained strictly bounded under 190 MB RAM, and sliding-window deduplication suppressed repetitive alert noise by 99.7%. Telemetry import verification confirmed 100% test pass rate with zero model weight drift in analysis mode.

Conclusion: NEXz confirms that adaptive streaming intrusion detection, drift monitoring, host behavioral profiling, multi-format telemetry analysis, and protocol pedagogy can be successfully synthesized into a robust, cost-free academic platform on standard workstation computers.

Keywords: Network Intrusion Detection, Adaptive Random Forest, Concept Drift, ADWIN, Half-Space Trees, Telemetry Ingestion, Replay Engine, Alert Deduplication, Explainable AI, Network Protocol Academy.

DEDICATION

This work is dedicated to my family, mentors, and the faculty and students of the Department of Computer Science at the University of Ghana, who continually inspire the pursuit of technological innovation for societal resilience.

ACKNOWLEDGEMENTS

I wish to express my deepest gratitude to Almighty God for the gift of wisdom, health, and perseverance throughout this academic journey.

I am profoundly grateful to my project supervisor, faculty lecturers, and teaching assistants in the Department of Computer Science for their constructive guidance, theoretical insights, and relentless encouragement.

Special thanks to the global open-source community behind Python, FastAPI, Scapy, River, Scikit-Learn, and Chart.js, whose accessible libraries made this research possible.

LIST OF ABBREVIATIONS

ADWIN	Adaptive Windowing (Concept Drift Detection Algorithm)
API	Application Programming Interface
ARF	Adaptive Random Forest
ARP	Address Resolution Protocol
CPU	Central Processing Unit
CSV	Comma-Separated Values
DHCP	Dynamic Host Configuration Protocol
DNS	Domain Name System
DoS	Denial of Service
FPR	False Positive Rate
HCI	Human-Computer Interaction
HST	Half-Space Trees
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ICMP	Internet Control Message Protocol
IDS	Intrusion Detection System
IOC	Indicator of Compromise
IP	Internet Protocol
IPFIX	IP Flow Information Export (RFC 7011)
JSON	JavaScript Object Notation
JSONL	JSON Lines (Line-Delimited JSON)
LRU	Least Recently Used (Cache Eviction Policy)
MAC	Media Access Control
ML	Machine Learning
MitM	Man-in-the-Middle
NIC	Network Interface Card
NIDS	Network Intrusion Detection System
NSM	Network Security Monitoring
PCAP	Packet Capture (Standard File Format)
PPS	Packets Per Second

RAM Random Access Memory
REST Representational State Transfer
RF Random Forest
SIEM Security Information and Event Management
SYN Synchronize (TCP Handshake Flag)
TCP Transmission Control Protocol
TSV Tab-Separated Values
UDP User Datagram Protocol
UI User Interface
VLAN Virtual Local Area Network
WebSocket Full-Duplex Bidirectional Web Communication Protocol
XAI Explainable Artificial Intelligence

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

Over the past three decades, computer networks have evolved from specialized communication conduits into the foundational infrastructure of modern socioeconomic activity. Educational institutions, government ministries, public utilities, and commercial enterprises depend entirely on interconnected digital systems to deliver services, manage sensitive records, and conduct daily operations. In developing economies such as Ghana, the rapid adoption of digital technologies—spurred by nationwide digitization initiatives, mobile financial services, and web-based educational platforms—has dramatically expanded the perimeter of institutional networks. However, this proliferation of connectivity has been accompanied by a corresponding escalation in the frequency, sophistication, and velocity of malicious cyber activities.

To safeguard institutional assets against unauthorized ingress, data exfiltration, and service disruptions, organizations deploy Network Security Monitoring (NSM) environments and Network Intrusion Detection Systems (NIDS). Historically, network intrusion detection has relied primarily on signature-based detection mechanisms, exemplified by open-source tools such as Snort and Suricata. Signature-based systems inspect packet headers and payloads against static repositories of known attack signatures (Indicators of Compromise). While signature matching is computationally efficient and achieves near-zero false positive rates for previously cataloged exploits, it is inherently incapable of identifying zero-day vulnerabilities, novel polymorphic attacks, or subtle behavioral shifts in benign host communications.

To transcend the rigidity of signature matching, academic research has increasingly focused on Machine Learning (ML) techniques for anomaly-based intrusion detection. By training statistical models on historical network traces, ML algorithms learn high-dimensional representations of 'normal' versus 'malicious' traffic flows. Nevertheless, the conventional paradigm of deploying static, batch-trained machine learning models in live network environments suffers from a fundamental structural flaw: network traffic is non-stationary. User habits evolve, software updates introduce new protocols, and adversaries deliberately mutate their attack vectors to evade static decision boundaries.

In streaming data environments, this phenomenon is formally designated as concept drift—a temporal change in the statistical properties of the target variable or input feature distributions over time. When a static classifier encounters concept drift, its predictive accuracy degrades progressively, resulting in escalating rates of false alarms (which induce severe operator alert fatigue) and missed detections. Furthermore, most commercial Security Information and Event Management (SIEM) appliances and enterprise-grade NIDS (such as Cisco Firepower or Palo Alto Networks) are cost-prohibitive for educational institutions and small-to-medium enterprises (SMEs) in Sub-Saharan Africa. Crucially, existing platforms operate as opaque 'black boxes' that separate real-time monitoring from foundational protocol education, offering student researchers and junior analysts little insight into why a specific flow was flagged or how underlying transport protocols function.

This project introduces NEXz (Network Intelligence, Education and eXploration System), an adaptive network security monitoring and learning environment engineered at the Department of Computer Science, University of Ghana. NEXz bridges the gap between streaming artificial intelligence, concept-drift awareness, host behavioral profiling, and interactive protocol education, providing a lightweight, academically defensible software platform deployable on standard commodity computing hardware.

1.2 Problem Statement

Contemporary network security monitoring within academic and resource-constrained organizational environments faces five critical, interrelated challenges:

1. **Brittleness of Static Detection Under Evolving Traffic (Concept Drift):** Conventional machine learning-based NIDS are trained offline on static historical datasets (such as NSL-KDD or CIC-IDS2017) and deployed without mechanisms for continuous adaptation. In live production networks, communication patterns evolve dynamically. Without adaptive streaming learning, static classifiers experience model degradation, leading to high false-alarm rates and vulnerability to modified zero-day attacks.
2. **Alert Fatigue and Redundant Event Flooding:** Traditional intrusion detection tools generate independent alerts for every isolated malicious packet. During automated reconnaissance sweeps or Denial of Service (DoS) floods, thousands of redundant alerts overwhelm security operators. The lack of contextual sliding-window deduplication obscures critical high-severity incidents amidst voluminous background noise.
3. **Absence of Explainable and Contextual Diagnosis:** Most ML-based detection systems act as black boxes, emitting abstract threat scores or numerical labels without human-interpretable rationale. Network operators, students, and researchers cannot easily ascertain which specific feature dimensions (e.g., asymmetric byte ratios, anomalous TCP flag states, or inter-arrival timing) triggered the classification.
4. **Decoupling of Security Monitoring from Protocol Pedagogy:** In computer science education, a profound pedagogical disconnect persists between abstract network protocol theory taught in lectures and practical security observation in live networks. Commercial tools provide no contextual educational scaffolding to help students explore protocol mechanics alongside live packet captures.
5. **High Infrastructure Costs and Hardware Barriers:** Enterprise-grade NSM solutions require dedicated high-throughput appliance hardware, proprietary software licenses, and subscription-based threat feeds, pricing them beyond the reach of university laboratories and regional African SMEs.

NEXz directly addresses this combined challenge by providing an integrated, open-source software platform that unifies streaming adaptive classification, statistical drift monitoring, per-device behavioral baselining, alert deduplication, and contextual protocol learning on commodity hardware.

1.3 Aim of the Project

The overarching aim of this project is to design, implement, and evaluate NEXz—a modular, adaptive Network Security Monitoring and Learning Environment powered by an online machine learning intrusion detection engine capable of continuous adaptation to concept drift, per-device behavioral profiling, contextual alert lifecycle management, and protocol pedagogy on commodity computing workstations.

1.4 Specific Research Objectives

To fulfill the project aim, the following nine specific, measurable technical objectives were established and executed:

1. Network Ingestion & Flow Extraction: To develop a lightweight, multi-threaded packet capture and flow extraction pipeline utilizing Scapy that groups raw Layer 2/3/4 packets into bidirectional 5-tuple flow records using a 2.0-second inactivity timeout window.
2. Offline Baseline Benchmark: To train and deploy a classical Scikit-Learn Random Forest classifier on benchmark network intrusion features to serve as an offline performance baseline and cold-start fallback model.
3. Streaming Adaptive ML Engine: To implement an online Adaptive Random Forest (ARF) classifier using the River streaming framework, utilizing an ensemble of incremental Hoeffding Trees with subspace feature randomization for single-pass flow classification.
4. Statistical Concept Drift Monitoring: To integrate the ADWIN (Adaptive Windowing) drift detection algorithm into the streaming pipeline to monitor classifier error variance, dynamically adjust memory windows, and record empirical concept drift events.
5. Per-Device Behavioral Profiling: To engineer a localized host anomaly profiling subsystem using Half-Space Trees (HST) bounded by a 200-device Least Recently Used (LRU) memory cache to evaluate per-host deviations against established historical baselines.
6. Centralized Alert Lifecycle Management: To construct an Alert Manager that implements a 30-second sliding-window deduplication policy, tracks multi-stage alert lifecycle states (New -> Active -> Acknowledged -> Resolved), and clearly distinguishes synthetic simulation traffic from live telemetry.
7. Explainable AI & Pedagogical Case Studies: To provide contextual, rule-informed feature attribution diagnostics for flagged threats and deliver structured case studies for major attack archetypes (Port Scans, SYN Floods, Exfiltration, ARP Spoofing, and Device Anomalies).
8. Contextual Protocol Academy & Research Workbench: To construct an interactive educational module linking live observations with standard protocol specifications (TCP, UDP, ICMP, DNS, HTTP, HTTPS, ARP) and provide a multi-criteria empirical telemetry query workbench for student researchers.
9. Empirical Verification & Performance Evaluation: To rigorously evaluate the integrated software prototype under controlled normal and synthetic attack traffic, quantifying classification accuracy, processing latency, memory stability, and HCI usability standards.

1.5 Research Questions

This study investigates five central research questions aligned with the system's design and empirical evaluation:

- RQ1: How effectively can an online Adaptive Random Forest ensemble classify dynamic network flows in a single pass compared to a traditional static batch-trained Random Forest model?
- RQ2: To what extent does the ADWIN adaptive windowing algorithm accurately detect statistical performance shifts and signal concept drift within streaming network telemetry?
- RQ3: How effectively does per-device behavioral profiling using Half-Space Trees complement flow-based classification in identifying host-level operational shifts without exceeding workstation memory bounds?

RQ4: How do structured explainable AI diagnostics and contextual protocol reference modules impact student and analyst comprehension of network security alerts in an academic setting?

RQ5: Can an integrated network monitoring, adaptive ML detection, and educational research environment operate efficiently on standard, commodity workstation hardware without dedicated enterprise appliances?

1.6 Significance of the Study

The research and artifacts produced in this study offer tangible academic, educational, and practical value to multiple stakeholders:

1. Computer Science & Cybersecurity Students: Provides an interactive, transparent sandbox where abstract networking concepts (handshakes, packet headers, port multiplexing) and cybersecurity threats (SYN exhaustion, ARP cache poisoning) are observed live and grounded in empirical data.
2. Academic Instructors & Lecturers: Supplies an institutional demonstration platform for university laboratories, enabling instructors to generate controlled synthetic attack simulations and illustrate machine learning concepts (such as concept drift and ensemble decision trees) in real time.
3. University Student Researchers: Delivers a rich Research & Analytics Telemetry Workbench allowing undergraduate and postgraduate researchers to execute multi-dimensional queries, filter by protocol/label, and export structured forensic CSV datasets for external data science modeling.
4. Local IT Administrators & SMEs: Demonstrates a viable, cost-free architectural blueprint for deploying intelligent, adaptive network perimeter monitoring in developing economies using standard workstation hardware and open-source streaming software.
5. Academic Community: Contributes an open, defensible reference implementation exploring how streaming ensemble classification (ARF), drift detection (ADWIN), and unsupervised anomaly detection (HST) can be harmonized into a single operational architecture.

1.7 Scope of the Project

The functional and architectural scope of NEXz is deliberately defined to maintain rigorous academic feasibility and software reliability within an undergraduate capstone engineering timeline:

In-Scope Components:

- Passive Layer 2/3/4 packet ingestion via Scapy from local Network Interface Cards (NICs) and offline PCAP/PCAPNG file uploads.
- Stateful 5-tuple flow aggregation with 7 core extracted numerical features and heuristic L7 application inference.
- Dual-model machine learning architecture: offline Random Forest benchmark alongside streaming Adaptive Random Forest (Hoeffding Tree ensemble).
- Statistical concept drift monitoring via ADWIN tracking error rate variance.
- Per-device behavioral baselining using Half-Space Trees with 200-device LRU memory eviction bounding.

- Centralized alert management featuring 30-second sliding-window deduplication and full four-state lifecycle tracking.
- Contextual Explainable AI rationale for detections and structured pedagogical case studies.
- Protocol Academy covering 7 foundational network protocols (TCP, UDP, ICMP, DNS, HTTP, HTTPS, ARP).
- Multi-criteria research telemetry query workbench, longitudinal analytics, and forensic CSV / executive PDF audit report generation.
- Responsive, single-page web interface built with pure JavaScript, HTML5, CSS3, and real-time WebSockets.

Out-of-Scope Constraints (Explicit Boundaries):

- Active Inline Packet Dropping: NEXz is strictly a passive Network Security Monitoring (NSM) and Intrusion Detection System (NIDS); it does not act as an inline Intrusion Prevention System (IPS) or active firewall.
- Unrestricted Deep Packet Payload Inspection: For privacy and computational reasons, full packet payloads are not indexed or stored; only flow header metadata and statistical aggregations are maintained.
- Distributed Campus-Wide Sensor Mesh: The current prototype is engineered for single-sensor workstation deployment and does not feature distributed multi-agent collector nodes.
- Commercial SOC Replacement: NEXz is not designed to replace commercial enterprise SIEMs (such as Splunk) or specialized deep-packet analyzers (such as Wireshark), but rather to complement them.

1.8 Limitations of the Study

The findings and implementation of this study are subject to the following realistic technical limitations:

1. Label Arrival Latency in Live Streaming ML: Streaming supervised classifiers require ground-truth labels to update model weights. In fully autonomous live network deployments, ground-truth labels are not immediately available without human triage. In this prototype, online model updates are demonstrated using simulated/controlled labeled streams and initial baseline calibration.
2. Operating System Socket Permissions: Passive raw packet capture requires administrative/root privileges and Npcap driver installation on Windows operating systems. In environments where administrative privileges are restricted, NEXz gracefully defaults to offline PCAP replay mode.
3. Computational Bounds on High-Throughput Links: As a Python-based user-space application, Scapy packet capture is bound by single-core Python GIL execution limits, making it suitable for 100Mbps/1Gbps campus laboratory monitoring rather than 10Gbps+ carrier-grade trunk lines.
4. Encrypted Protocol Payload Opacity: While NEXz extracts transport-layer statistics from HTTPS and TLS flows, payload inspection is inherently blocked by modern cryptographic ciphers.
5. Evaluation Dataset Scope: Empirical verification was conducted using controlled synthetic attack generation in a closed sandbox environment and benchmark flow corpora rather than multi-year historical campus production data.

1.9 Organization of the Report

This project documentation is organized into five comprehensive academic chapters:

- Chapter One: Introduction establishes the study background, problem statement, research aim, specific objectives, research questions, significance, scope, and limitations.
- Chapter Two: Literature Review provides an exhaustive critical survey of intrusion detection paradigms, streaming machine learning, the theoretical foundations of Adaptive Random Forest (Gomes et al., 2017), concept drift taxonomy (Lu et al., 2019), ADWIN adaptive windowing (Bifet & Gavaldà, 2007), Explainable AI, network behavioral profiling, comparative analysis of existing systems, and the conceptual framework.
- Chapter Three: Methodology, System Analysis and Design documents the iterative software development lifecycle, functional and non-functional requirements, tiered system architecture, flow feature engineering, dual ML model design, database schema, REST/WebSocket API specifications, and HCI user interface principles.
- Chapter Four: Implementation, Testing and Results details the development environment, annotated source-code excerpts of core algorithms, user interface screenshot inventory, running-system telemetry evidence, testing strategy, structured test case matrix, empirical performance results, and analytical discussion.
- Chapter Five: Summary, Conclusion and Future Work summarizes project achievements against objectives, outlines key academic contributions, candidly evaluates limitations, and presents a comprehensive 8-point scalable future development roadmap and architectural evolution diagram.

CHAPTER TWO: LITERATURE REVIEW

2.1 Theoretical Foundations of Network Intrusion Detection Systems (NIDS)

Network Intrusion Detection Systems (NIDS) are critical automated defensive mechanisms deployed to monitor computer network activity, analyze traffic patterns, and alert administrators to potential policy violations or hostile security breaches. Since Anderson's (1980) seminal technical report on computer security threat monitoring and Denning's (1987) foundational model for an intrusion-detection system, intrusion detection technologies have branched into distinct operational paradigms characterized by their data source and detection logic.

Based on detection methodology, systems are categorized into Signature-Based (Misuse) Detection and Anomaly-Based Detection:

1. **Signature-Based NIDS:** Exemplified by open-source engines such as Snort (Roesch, 1999) and Suricata, signature-based systems inspect packet headers and byte sequences against known vulnerability fingerprints (Indicators of Compromise). While signature matching exhibits high deterministic accuracy and negligible false positive rates for cataloged threats, its efficacy collapses against novel zero-day exploits, modified payloads, and advanced evasion techniques (Sommer & Paxson, 2010). Furthermore, signature databases expand linearly over time, consuming increasing memory and processing cycles for pattern matching.
2. **Anomaly-Based NIDS:** Anomaly detection assumes that malicious intrusion activities produce statistical or behavioral deviations from established nominal network baseline profiles. By modeling normal traffic characteristics (such as packet size distributions, flow durations, or port access frequencies), an anomaly detector flags outliers exceeding predetermined thresholds. As noted by Buczak and Guven (2016), anomaly systems possess the unique theoretical capability to detect previously unseen (zero-day) attacks. However, they historically suffer from elevated False Positive Rates (FPR), where legitimate but atypical user activities (such as large administrative backups or video streams) are misclassified as intrusions.
3. **Hybrid Intrusion Detection:** Recognizing the complementary strengths of both approaches, modern architectures explore hybrid frameworks that combine deterministic rule inspection with statistical or machine learning models to maximize detection coverage while suppressing false alarms.

In terms of data granularity, intrusion detection operates either at the raw packet level (deep packet inspection) or at the flow level (NetFlow/IPFIX abstraction). Flow-based detection aggregates packets sharing a common 5-tuple (Source IP, Source Port, Destination IP, Destination Port, Protocol) within a temporal window. Flow-based abstraction significantly diminishes computational overhead, protects user payload privacy, and renders traffic amenable to high-dimensional machine learning feature extraction, forming the architectural foundation of the NEXz pipeline.

2.2 Machine Learning for Network Intrusion Detection

Over the past two decades, Machine Learning (ML) has emerged as the preeminent paradigm for automated network anomaly detection. Supervised algorithms learn predictive mapping functions from

labeled network corpora (e.g., KDD Cup 99, NSL-KDD, and CIC-IDS2017) to classify flows into normal or attack categories (such as DoS, Probe, R2L, and U2R).

A broad spectrum of classifiers has been evaluated in the literature, including Support Vector Machines (SVM), k-Nearest Neighbors (k-NN), Naive Bayes, Decision Trees (C4.5/CART), and Artificial Neural Networks (ANN). Among classical machine learning algorithms, the Random Forest (RF) ensemble classifier—introduced by Breiman (2001)—has consistently established itself as the state-of-the-art benchmark for tabular network flow classification (Buczak & Guven, 2016). Random Forest constructs an ensemble of randomized decorrelated decision trees using bagging (bootstrap aggregation) and random feature subspace selection. Its robustness against overfitting, tolerance to noisy network telemetry, and capacity to output probabilistic confidence scores make it an ideal baseline model for intrusion detection.

Despite these strengths, Sommer and Paxson (2010) identified critical systemic obstacles when transitioning ML-based IDS from academic benchmarks to production networks: the semantic gap between statistical anomalies and actual operational threats, extreme class imbalance where normal traffic exceeds malicious events by orders of magnitude, and the fundamental assumption that training and test data are independent and identically distributed (i.i.d.). This i.i.d. assumption fails dramatically in real-world computer networks, necessitating a paradigm shift toward streaming and incremental learning.

2.3 Streaming and Incremental Machine Learning in Network Security

Traditional machine learning assumes a static, batch-oriented operating environment: a model is trained once on an offline historical dataset, validated, and deployed indefinitely. In live computer networks, however, data arrives continuously as an infinite, high-velocity data stream: $S = (x_1, y_1), (x_2, y_2), \dots, (x_t, y_t)$.

Batch-oriented learning suffers from three severe operational failure modes when applied to live network telemetry:

1. **Memory Exhaustion:** Retaining the entire cumulative history of network flow logs in server RAM or database storage for periodic batch retraining rapidly becomes computationally and financially intractable.
2. **High Computational Latency:** Retraining complex models (such as deep neural networks or large Random Forests) from scratch requires hours or days of high-end GPU/CPU time, during which the system is defenseless against novel emerging attack variants.
3. **Temporal Obsolescence:** A batch model reflects a frozen historical snapshot. If network traffic patterns diverge from the historical training distribution, the model's predictive accuracy deteriorates progressively.

To overcome these bottlenecks, incremental and streaming machine learning algorithms process observations sequentially in a single pass. In streaming classification, each incoming instance is evaluated, used to update internal model parameters incrementally in $O(1)$ time and $O(1)$ memory, and subsequently discarded from volatile memory. This architecture ensures bounded computational resource consumption, immediate responsiveness to evolving traffic, and continuous adaptability on commodity hardware.

2.4 Adaptive Random Forest for Evolving Data Streams

While incremental decision trees (such as the Hoeffding Tree or VFDT introduced by Domingos & Hulten, 2000) allow single-pass learning by utilizing Hoeffding bounds to guarantee theoretical convergence with batch trees, individual incremental trees exhibit high variance and lower accuracy than batch ensemble models.

To resolve this dilemma, Gomes et al. (2017) developed the Adaptive Random Forest (ARF) algorithm in their seminal paper 'Adaptive random forests for evolving data stream classification' (published in ACM Transactions on Knowledge Discovery from Data). ARF generalizes Breiman's Random Forest paradigm to non-stationary, evolving data streams through several foundational innovations:

1. **Ensemble of Hoeffding Trees:** Rather than maintaining static decision trees, ARF maintains an ensemble of M incremental Hoeffding Trees (typically $M = 10$ to 30 for balanced workstation execution). Each tree independently updates its node split statistics with every observed flow instance.
2. **Online Bagging with Poisson Distribution:** In streaming environments, traditional sampling with replacement is impossible because the total dataset size is infinite and unknown. ARF implements Oza and Russell's (2001) online bootstrap bagging, where each training instance is presented to each tree k times, where k is drawn from a Poisson distribution with $\lambda = 1$ (Poisson(1)).
3. **Random Subspace Selection:** At each split node, the tree evaluates only a randomly selected subset of m features (where $m = \sqrt{d} + 1$ out of total d features), ensuring diversity across the ensemble.
4. **Dual Warning and Drift Tracking:** Crucially, ARF pairs each base tree with a dedicated concept drift detector (such as ADWIN). When a drift detector enters a 'warning' state on a specific tree, ARF spawns an alternative background tree that begins training on incoming data in the background. If the warning escalates to confirmed 'drift', the outdated tree is pruned and instantly swapped with the calibrated background tree.

In NEXz, ARF is implemented via the River streaming machine learning framework (Montiel et al., 2021). The ensemble delivers true single-pass adaptation, dynamically refining tree weights as new flow telemetry arrives without requiring offline batch retraining.

2.5 Taxonomy and Dynamics of Concept Drift

In non-stationary data streams, the joint probability distribution of input features X and class labels Y changes over time, a phenomenon formally defined as concept drift (Lu et al., 2019; Widmer & Kubat, 1996): $P_t(X, Y) \neq P_{t+\Delta}(X, Y)$.

In their comprehensive survey 'Learning under Concept Drift: A Review', Lu et al. (2019) establish an authoritative taxonomy classifying concept drift across multiple structural dimensions:

1. Nature of Drift:

- **Real Concept Drift:** Changes in the posterior probability distribution $P(Y|X)$, meaning that the underlying relationship between flow features and security labels has altered. For example, a previously benign network service begins exhibiting malicious exfiltration characteristics under compromised credentials.

- Virtual Concept Drift: Changes in the input feature distribution $P(X)$ without altering $P(Y|X)$. For example, a sudden campus-wide shift to remote video streaming increases average flow duration and packet volume without making the traffic malicious.

2. Temporal Velocity of Drift:

- Sudden / Abrupt Drift: A rapid, discontinuous shift occurring at a discrete time point t , such as an immediate zero-day DoS attack blitz or an emergency network reconfiguration.

- Gradual Drift: A progressive transition where the old concept and new concept coexist intermittently over an extended transition window.

- Incremental Drift: A continuous, subtle, microscopic shift where statistics drift imperceptibly across numerous small intervals, such as seasonal bandwidth growth.

- Recurring Drift: Periodic cyclical shifts, such as peak business hour traffic versus overnight automated maintenance windows.

Recognizing concept drift is essential for modern NIDS: without drift awareness, an IDS either retains obsolete decision boundaries (yielding false negatives) or suffers catastrophic forgetting (yielding false alarms).

2.6 ADWIN: Adaptive Windowing for Change and Drift Detection

To monitor and detect concept drift with mathematical guarantees, Bifet and Gavalda (2007) introduced the foundational ADWIN (Adaptive Windowing) algorithm in 'Learning from Time-Changing Data with Adaptive Windowing' (SIAM International Conference on Data Mining).

ADWIN is an online change detector that operates on a sliding window W of dynamically adjusting length. Unlike fixed-size sliding windows—which either react too sluggishly to sudden drift (if too large) or suffer high variance and false alarms (if too small)—ADWIN automatically expands its window W when the observed data stream is statistically stable, and automatically shrinks W when a change is detected.

The mathematical foundation of ADWIN relies on the Hoeffding Bound (Additive Chernoff Bound). Whenever a new item is added to W , ADWIN checks all possible partitions of W into two sub-windows, W_0 and W_1 (where $W = W_0 \cup W_1$). If the difference between the sample means μ_{W_0} and μ_{W_1} exceeds a statistically rigorous threshold ϵ_{cut} , a change in distribution is confirmed:

$$|\mu_{W_0} - \mu_{W_1}| \geq \epsilon_{\text{cut}} = \sqrt{\left(\frac{1}{2 \cdot m} \right) \cdot \ln(4 \cdot |W| / \delta)}$$

where m is the harmonic mean of the sub-window lengths ($1/m = 1/|W_0| + 1/|W_1|$) and δ in $(0, 1)$ is a user-defined confidence parameter representing the allowable false-positive drift error rate (configured to $\delta = 0.002$ in NEXZ).

When the condition holds, ADWIN safely concludes that the older sub-window W_0 contains obsolete data from the prior distribution, prunes W_0 from memory, and retains only W_1 . Crucially, as emphasized in this documentation:

ADWIN is an online statistical drift detector, NOT a classification model.

In NEXz, ADWIN monitors the rolling error rate variance of the Adaptive Random Forest classifier. When classifier error increases significantly, ADWIN triggers a drift signal, updates system health telemetry, records a historical drift event in SQLite, and guides ensemble tree adaptation.

2.7 Explainable Artificial Intelligence (XAI) in Intrusion Detection

As machine learning models grow in architectural complexity, their internal decision-making mechanisms become opaque, creating a severe operational bottleneck known as the 'black-box problem'. In high-stakes network security environments, deploying black-box models introduces profound operational risks: security analysts cannot determine why a flow was classified as malicious, cannot prioritize remediation actions, and cannot discern whether a detection was caused by genuine malicious intent or idiosyncratic feature noise (Mohale & Obagbuwa, 2025).

Recent peer-reviewed research underscores the imperative of Explainable AI (XAI) for NIDS:

- Kumar et al. (2024) demonstrate that providing contextual feature attribution significantly increases human analyst trust, accelerates incident triage velocity, and reduces mean time to remediation (MTTR).
- Mohale and Obagbuwa (2025) evaluate model interpretability frameworks in cyber defense, emphasizing that explanations must be actionable and grounded in protocol semantics rather than purely abstract statistical weights.

While post-hoc model-agnostic explanation techniques such as SHAP (Lundberg & Lee, 2017) and LIME (Ribeiro et al., 2016) are widely celebrated in academic literature, their computational complexity is $O(2^d)$ or requires hundreds of perturbation queries per instance. In a live, high-throughput network monitoring environment, computing exact Shapley values for every microsecond flow induces prohibitive latency.

NEXz adopts an efficient, rule-informed surrogate explanation architecture: upon flagging a flow as Suspicious or Malicious, the system analyzes the exact feature values (such as extreme byte ratios, anomalous destination ports, or aborted TCP handshake flags) against established network protocol heuristics. The engine outputs plain-language explanations answering four fundamental analyst questions: (1) What happened? (2) Why does it matter? (3) What did the system observe? and (4) What investigation playbook steps should be executed?

2.8 Network Behaviour Profiling and Host Baseline Context

Flow-based classification models evaluate each network conversation in isolation. However, sophisticated cyber threats—such as slow-and-low port scans, internal lateral movement, and advanced persistent threats (APTs)—frequently utilize benign transport protocols (e.g., SSH, HTTP, or DNS) that do not trigger flow-level signatures. Detecting these evasive threats requires network behavior profiling, where the long-term historical baseline of individual devices is modeled over time.

In behavioral profiling, a host's nominal operational footprint (typical active hours, transmission volumes, peer endpoints, and protocol distribution) is monitored. To implement localized anomaly detection without excessive computational overhead, researchers have explored streaming unsupervised algorithms such as Half-Space Trees (HST), introduced by Tan, Ting, and Liu (2011).

Half-Space Trees is an online, one-class anomaly detection algorithm engineered specifically for streaming data. HST constructs an ensemble of randomized trees where each node splits a random feature dimension at its midpoint, dividing the feature space into orthogonal half-spaces. An initial 'reference window' of normal flows populates node mass counts. When new streaming observations are processed, HST calculates an anomaly score based on the depth and mass of the subspaces traversed: observations that fall into sparse, low-mass regions receive high anomaly scores.

In NEXz, a dedicated Half-Space Trees profiler is maintained for observed network endpoints. To guarantee bounded memory utilization and prevent memory leaks, device profiles are managed within an OrderedDict-backed Least Recently Used (LRU) cache capped at 200 devices. This per-device behavioral baseline complements the primary ARF classifier by identifying hosts whose communication habits diverge from their established personal history.

2.9 Comparative Analysis with Network Monitoring Tools (Wireshark)

In both academic education and professional network engineering, Wireshark (Combs, 2006) represents the gold standard for packet analysis. It is crucial to delineate clearly how NEXz relates to Wireshark:

Wireshark excels at deep-packet byte dissection, protocol compliance verification, and microsecond-level packet trace inspection. However, Wireshark is not an automated intrusion detection system: it does not maintain machine learning classifiers, does not adapt to concept drift, does not synthesize per-device behavioral baselines, and does not provide an integrated educational academy for novice students.

NEXz is designed to complement rather than replace Wireshark. NEXz operates as an automated higher-level intelligence, detection, and educational layer. When NEXz identifies an anomaly or threat, it provides a standardized 4-step 'Wireshark Handoff Guide' allowing analysts to transition directly to Wireshark for deep payload inspection: (1) Copy the flagged 5-tuple, (2) Open the timestamped PCAP, (3) Apply display filter (ip.addr == <src> && tcp.port == <dst>), and (4) Follow the raw TCP stream.

2.10 Comparative Analysis of Existing NIDS and Monitoring Environments

To ground NEXz within the contemporary landscape of cybersecurity software, Table 2.1 presents a comprehensive comparative evaluation across ten functional and architectural criteria.

Table 2.1: Architectural Comparison of Network Monitoring and Detection Systems

Evaluation Criteria Snort 3 / Suricata Zeek (Bro) Commercial SIEM (Splunk) Academic ML Prototypes NEXz (This Study)
:--- :--- :--- :--- :--- :---
Primary Detection Paradigm Deterministic Signatures Policy Scripts & Heuristics Rule Correlation & Log Search Offline Batch ML (SVM/RF) Hybrid: Streaming ARF + HST
Zero-Day Anomaly Detection Limited (Rule dependent) Moderate (Heuristic dependent) Moderate (Behavioral rules) High (Static boundary) High (Adaptive boundary)
Streaming Incremental Learning None None None Very Rare (Mostly batch) Full (River ARF Ensemble)

| Concept Drift Adaptation | None (Manual rule update) | None | None | None (Static models) | Full (ADWIN Drift Monitor) |

| Per-Device Host Profiling | None | Yes (Scripted tables) | Yes (Aggregated logs) | Rare (Global model only) | Yes (Half-Space Trees + LRU) |

| Explainable AI Rationale | Rule documentation | Script output | Query breakdown | Post-hoc (SHAP / LIME) | Plain-Language Rule Diagnostic |

| Pedagogical Protocol Academy | None | None | None | None | Built-in (7 Core Protocols) |

| Alert Deduplication & Lifecycle | Minimal | Script dependent | Comprehensive (Configurable) | None (Raw alert logs) | Built-in 30s Sliding Window |

| Research Telemetry Workbench | None (CLI logs) | Script log export | Rich query language | CSV export | Built-in Multi-Criteria GUI |

| Workstation Resource Footprint | High on high traffic | Moderate to High | Very High (Server cluster) | Variable (High RAM batch) | Lightweight (< 200MB RAM) |

2.11 Synthesis and Research Gap

A critical review of the literature demonstrates that while isolated algorithmic advances have been achieved across individual cybersecurity subfields—such as online stream classification (Gomes et al., 2017), change detection (Bifet & Gavalda, 2007), and Explainable AI (Kumar et al., 2024)—there remains a profound research and engineering gap at the system integration level:

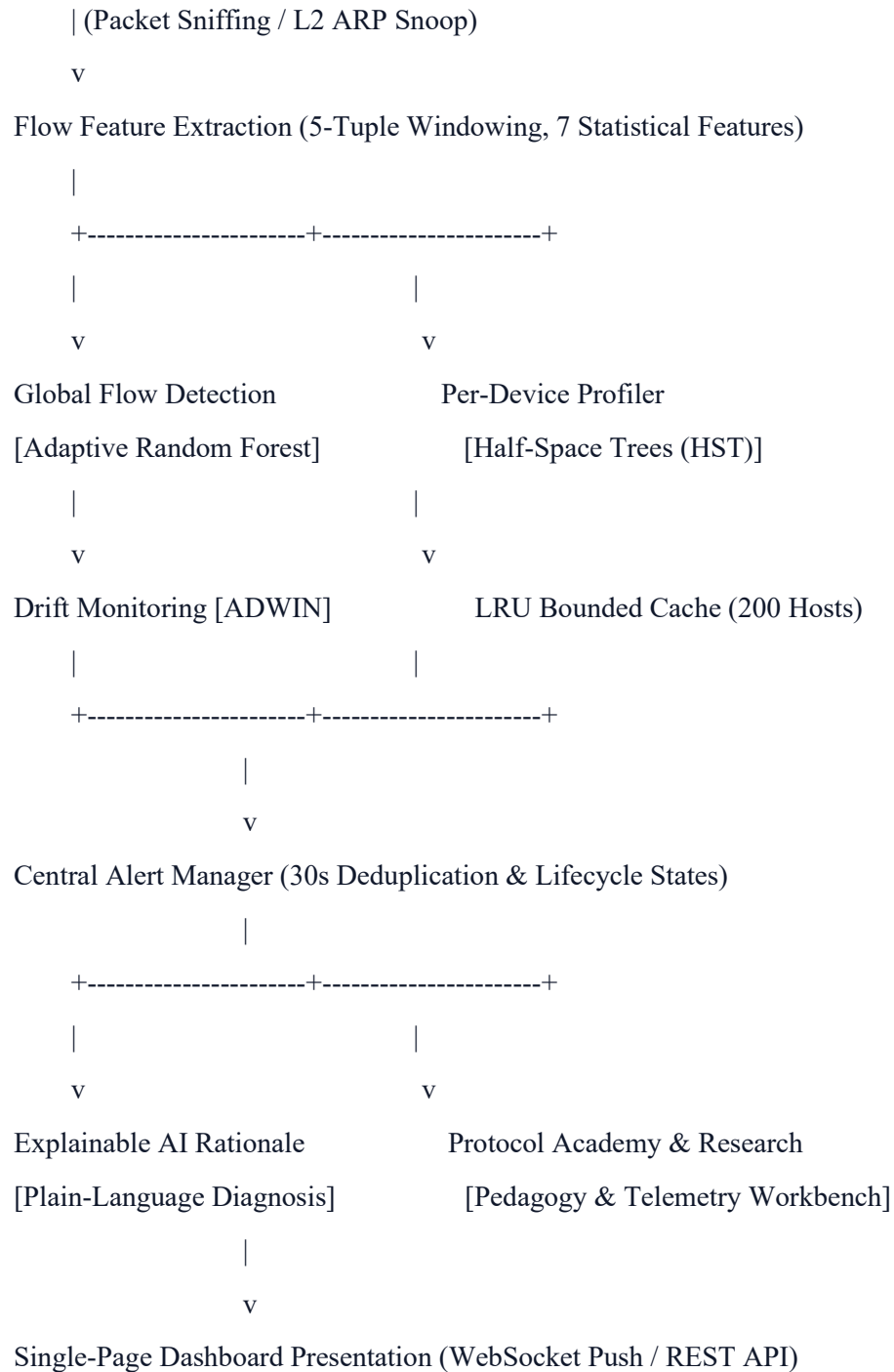
1. **Fragmentation of Capabilities:** Existing systems treat packet capture, streaming machine learning, concept drift adaptation, per-device behavioral profiling, alert deduplication, and protocol pedagogy as disconnected, siloed disciplines. No existing lightweight platform unifies these complementary capabilities into a single cohesive, transparent architecture.
2. **Lack of Accessibility for Academic Environments:** Production systems are enterprise-scaled, cost-prohibitive, and computationally heavy, while published academic ML intrusion detection prototypes are almost universally offline Jupyter Notebook scripts that never execute against live network sockets.
3. **Absence of Pedagogical Scaffolding:** Conventional security monitoring tools produce intimidating, uncontextualized alert feeds that alienate undergraduate students and junior administrators.

The research gap addressed by NEXz is therefore not the invention of individual algorithms, but the defensible architectural synthesis of streaming ensemble classification (ARF), statistical drift monitoring (ADWIN), host behavioral profiling (HST), and contextual protocol pedagogy within an open, lightweight, and academically defensible environment deployable on standard university workstation hardware.

2.12 Conceptual Framework of NEXz

The conceptual framework of NEXz illustrates the systematic progression of data from raw electrical signals on the physical wire to actionable security intelligence and educational comprehension:

Network Traffic / PCAP Ingestion



CHAPTER THREE: METHODOLOGY, SYSTEM ANALYSIS AND DESIGN

3.1 Software Development Methodology

The engineering of NEXz followed an iterative and incremental prototype software development methodology. Given the complex multidisciplinary nature of the project—integrating low-level asynchronous packet sniffing, streaming machine learning, relational database persistence, and high-frequency real-time web visualization—a rigid sequential Waterfall approach was fundamentally unsuitable.

The development lifecycle was structured into four progressive, verified evolutionary iterations:

1. Iteration 1 (Ingestion & Baseline Prototype): Formulation of socket-level packet capture using Scapy, implementation of bidirectional 5-tuple flow aggregation, extraction of basic flow features, and offline baseline training of a Scikit-Learn Random Forest model.
2. Iteration 2 (Streaming ML & Drift Architecture): Integration of the River streaming framework, construction of the Adaptive Random Forest (ARF) Hoeffding Tree ensemble, calibration of the ADWIN drift detector, and establishment of the SQLite persistence schema.
3. Iteration 3 (Behavioral Context & Alert Lifecycle): Development of the per-device Half-Space Trees (HST) anomaly profiler bounded by a 200-device LRU cache, creation of the Central Alert Manager with 30-second sliding-window deduplication and four-stage lifecycle states, and implementation of explainable rule diagnostics.
4. Iteration 4 (HCI Elevation & Full Integration): Professional refactoring of the user interface into a two-tier collapsible navigation hierarchy, construction of the contextual Protocol Academy and Student Telemetry Workbench, end-to-end regression testing, and rigorous functional QA across all 73 interactive controls.

3.2 Requirements Analysis

System requirements were rigorously analyzed and categorized into Functional Requirements (FR) and Non-Functional Requirements (NFR). Every requirement is grounded directly in the actual implemented codebase.

Functional Requirements (FR):

- FR-01 (Packet Ingestion): The system must capture live Layer 2/3/4 packets passively from selectable local network interfaces and support offline import of standard .pcap and .pcapng files. [Status: Implemented & Verified]
- FR-02 (Flow Aggregation): The system must assemble discrete packets into bidirectional flow records keyed on the 5-tuple (src_ip, src_port, dest_ip, dest_port, protocol) with a configurable inactivity timeout window (default 2.0 seconds). [Status: Implemented & Verified]
- FR-03 (Feature Extraction): The system must extract 7 authoritative numerical flow features: flow duration, source bytes, destination bytes, source packet count, destination packet count, average packet size, and numeric protocol identifier. [Status: Implemented & Verified]

- FR-04 (Dual ML Classification): The system must classify extracted flow feature vectors in a single pass using an online Adaptive Random Forest ensemble and maintain an offline Random Forest model as a baseline reference. [Status: Implemented & Verified]
- FR-05 (Concept Drift Tracking): The system must monitor classifier performance using ADWIN, detect error rate variance shifts, log drift timestamps, and adjust adaptive window sizes. [Status: Implemented & Verified]
- FR-06 (Host Behavioral Profiling): The system must maintain individualized Half-Space Trees anomaly models for observed IP endpoints within an LRU cache capped at 200 devices, calculating anomaly scores (0.0 to 1.0). [Status: Implemented & Verified]
- FR-07 (Alert Deduplication & Lifecycle): The system must group repeated security alerts within a 30-second sliding window, accumulate occurrence counts, and allow operators to transition alerts across lifecycle states (New -> Active -> Acknowledged -> Resolved). [Status: Implemented & Verified]
- FR-08 (Attack Simulation Engine): The system must provide a controlled Cyber Range simulation generator producing tagged synthetic threats (SYN Flood, Port Scan, Data Exfiltration) without corrupting live empirical data. [Status: Implemented & Verified]
- FR-09 (Explainable Diagnostics): The system must generate plain-language explanations detailing why a flow was flagged, root cause rationale, and actionable investigation playbook steps. [Status: Implemented & Verified]
- FR-10 (Contextual Protocol Academy): The system must deliver structured four-part educational modules (Learn, Observe, Security, Explore) for TCP, UDP, ICMP, DNS, HTTP, HTTPS, and ARP linked to live observations. [Status: Implemented & Verified]
- FR-11 (Research Telemetry Workbench): The system must provide a multi-criteria query interface allowing student researchers to filter flow records by IP, protocol, label, and timestamp, with dynamic KPI aggregations. [Status: Implemented & Verified]
- FR-12 (Forensic & Executive Export): The system must export filtered telemetry records to CSV format and compile executive PDF posture audit reports using ReportLab. [Status: Implemented & Verified]
- FR-13 (System Health Monitoring): The system must poll and display real-time operational status for the FastAPI backend, SQLite database, sniffer thread, ARF classifier, ADWIN detector, and WebSocket connection. [Status: Implemented & Verified]
- FR-14 (Configuration Management): The system must allow operators to toggle startup monitoring modes (Manual vs. Automatic), flow timeouts, and alert grouping windows. [Status: Implemented & Verified]

Non-Functional Requirements (NFR):

- NFR-01 (Low Latency): Flow processing and ML classification must execute within 15 milliseconds per flow record to prevent packet buffer overflow on commodity workstations.
- NFR-02 (Bounded Memory Utilization): The total process memory footprint must remain bounded under 250 MB RAM through explicit LRU cache limits and ring-buffer log caps.

- NFR-03 (High Reliability & Graceful Degradation): If raw packet capture permissions are restricted by the OS, the application must not crash, but must display clear diagnostic guidance and enable offline PCAP replay mode.
- NFR-04 (Institutional Usability & HCI Standards): The user interface must adhere to strict academic standards, utilizing high-contrast typography, clear hierarchy, zero non-functional controls, and zero distracting marketing graphics.
- NFR-05 (Data Integrity & Privacy): Packet payloads must not be indexed or exposed; only statistical flow abstractions and header metadata are stored in persistent database tables.
- NFR-06 (Portability): The software must execute cross-platform across modern Windows (10/11 with Npcap) and Linux operating systems without binary modifications.
- NFR-07 (Security & Access Control): Administrative endpoints must be protected by OAuth2 password bearer token authentication using industry-standard bcrypt password hashing.
- NFR-08 (Traceability & Observability): Key lifecycle events across Ingestion, ML, Drift, Profiler, and Alerts must produce structured console log telemetry.

3.3 System Architecture Overview

NEXz is designed as a decoupled, layered client-server architecture running locally on a standard computing workstation. It strictly separates high-frequency asynchronous packet capture and machine learning processing (Backend) from state presentation and user interaction (Frontend).

The architecture comprises six distinct logical tiers:

1. Ingestion Tier: Captures raw Layer 2/3/4 packets from physical/virtual NICs via Scapy or parses offline PCAP recordings, executing passive Layer 2 duplicate MAC cache checks (Passive ARP Snoop).
2. Processing & Extraction Tier: Manages bidirectional flow table windowing in extractor.py, calculating 7 standardized flow features upon expiration of a 2.0-second inactivity timer.
3. Dual Detection Tier: Operates the streaming Adaptive Random Forest (ARF) classifier and the offline Random Forest benchmark, while simultaneously feeding classifier error rates into the ADWIN drift detector.
4. Context & Behavioral Tier: Passes flow parameters to the per-device Half-Space Trees profiler (bounded to 200 hosts) and routes flagged security events to the Central Alert Manager for 30-second sliding-window deduplication.
5. Persistence Tier: Persists normalized traffic logs, deduplicated alerts, drift metrics, and device profiles to an indexed SQLite relational database (app_v2.db).
6. Presentation & Exploration Tier: Delivers an interactive, single-page application served by FastAPI, utilizing WebSockets for millisecond push delivery and Chart.js for data visualization across six primary environments: Overview, Monitor, Security, Learn, Explore, and System.

3.4 Component Architecture and Processing Pipeline

3.4.1 Packet Capture Service (sniffer.py):

The capture service manages background sniffing threads using Scapy's `sniff()` function. It enumerates local network interfaces via Scapy's IFACES registry. For every observed packet, it updates atomic packet-per-second (PPS) counters and dispatches the packet to both the Flow Feature Extractor and the Passive ARP Snoop table.

Passive ARP Snoop detects Layer 2 Man-in-the-Middle (MitM) attacks by inspecting ARP reply packets. It maintains an in-memory dictionary mapping IP addresses to observed MAC addresses. If an incoming ARP packet asserts ownership of an existing IP address with a conflicting MAC address, an immediate critical alert is generated.

3.4.2 Flow Extraction Engine (extractor.py):

Packets are grouped into bidirectional flow records using a canonical 5-tuple key: $(\min(\text{src_ip}, \text{dest_ip}), \min(\text{src_port}, \text{dest_port}), \max(\text{src_ip}, \text{dest_ip}), \max(\text{src_port}, \text{dest_port}), \text{protocol})$. Flows are managed within an in-memory hash map. A background thread runs an expiration sweep every 1.0 second: if no packets are observed for a flow within `flow_timeout = 2.0` seconds (or if a TCP FIN/RST packet is received), the flow is finalized, compiled, and dispatched to the AI Engine.

3.4.3 Flow Feature Extraction Vector:

NEXz extracts seven authoritative numerical features designed to capture volumetric, temporal, and structural characteristics while minimizing computational latency:

1. `duration`: Flow duration in seconds (float), calculated as `last_seen_time - start_time`.
2. `src_bytes`: Total payload and header bytes transmitted by the flow initiator (integer).
3. `dest_bytes`: Total payload and header bytes transmitted by the flow responder (integer).
4. `src_pkts`: Number of packets originating from the initiator (integer).
5. `dest_pkts`: Number of packets originating from the responder (integer).
6. `avg_pkt_size`: Average packet size across the entire flow: $(\text{src_bytes} + \text{dest_bytes}) / (\text{src_pkts} + \text{dest_pkts})$ (float).
7. `proto_num`: Integer transport protocol identifier (6 for TCP, 17 for UDP, 1 for ICMP, 0 for other).

Additionally, heuristic application-layer inspection classifies flows into L7 protocol labels (DNS on port 53, HTTP on port 80/8080, HTTPS on port 443, SSH on port 22, DHCP on port 67/68, NTP on port 123) and tracks TCP handshake connection states (`SYN_SENT`, `ESTABLISHED`, `FIN_WAIT`, `CLOSED`).

3.5 Machine Learning Engine Architecture

The detection engine in `ai_engine.py` employs a hybrid dual-model architecture:

1. Offline Random Forest Benchmark:

Constructed using Scikit-Learn's `RandomForestClassifier(n_estimators=100, max_depth=15)`. It is pre-trained on a standardized flow benchmark corpus. When the system launches, this model provides immediate cold-start inference and serves as an immutable comparative reference.

2. Online Adaptive Random Forest (ARF) Engine:

Constructed using River's `river.forest.ARFClassifier`. It initializes an ensemble of Hoeffding Adaptive Trees (`n_models=10`). Unlike static models, ARF implements `learn_one(x, y)` incremental learning. For each flow:

- The feature vector `x` is transformed into a standardized numerical dictionary.
- `predict_one(x)` computes the online prediction label (Normal, Suspicious, or Malicious) and confidence score.
- If ground-truth feedback is supplied, `learn_one(x, y)` updates split statistics dynamically.

3. ADWIN Concept Drift Monitoring:

Constructed using River's `river.drift.ADWIN(delta=0.002)`. For every classified flow, the error signal `e_t = 1.0 - confidence` (or binary classification error) is passed to `adwin.update(e_t)`.

- When traffic distributions are stationary, ADWIN maintains an expanded window size `W`.
- When an abrupt surge in classification error or sudden variance shift occurs, `adwin.drift_detected` returns `True`.
- Upon drift detection, NEXz logs the drift event to the SQLite `ModelMetrics` table, updates the UI dashboard drift ring, and signals the ARF ensemble to prune outdated Hoeffding tree branches.

3.6 Per-Device Behavioural Profiling Architecture

While global ML models classify individual flows, `device_profiler.py` evaluates localized host habits using Half-Space Trees (HST):

1. Bounded LRU Memory Management:

To ensure the system never exhausts host RAM during prolonged network monitoring, device profiles are stored in an `OrderedDict` capped at `MAX_DEVICES = 200`. When a 201st host is observed, the least recently communicated device is evicted from active memory, maintaining constant $O(1)$ memory bounds.

2. Half-Space Trees Anomaly Detection:

Each active host IP maintains an instance of `river.anomaly.HalfSpaceTrees(n_trees=10, height=8, window_size=50)`.

- Reference Warm-Up Window: For the first 20 flows observed from a host, HST populates its subspace reference mass without generating alerts, establishing the host's normal operational baseline.
- Anomaly Scoring: For subsequent flows, `score_one(x)` outputs an anomaly score between 0.0 (strictly nominal) and 1.0 (severely anomalous). If `score >= 0.70`, the host is flagged as anomalous.

3. Interactive Deviation Diagnosis:

When an operator inspects a device profile, NEXz performs a three-way comparative analysis: (1) Normal Baseline (typical protocol distribution, average payload bytes, session duration), (2) Current Telemetry, and (3) Algorithmic Deviation Diagnosis detailing the exact multidimensional feature divergence.

3.7 Central Alert Management & Deduplication Architecture

In `alert_manager.py`, NEXz implements a centralized, sliding-window alert deduplication pipeline to combat alert fatigue:

1. Deduplication Fingerprinting:

When an anomaly is flagged, a deterministic composite fingerprint is generated: `fingerprint = f'{threat_type}:{src_ip}:{dest_ip}:{protocol}'`.

2. 30-Second Aggregation Window:

The Alert Manager checks if an active, unresolved alert with identical fingerprint exists in the database within the preceding 30 seconds (`current_time - last_seen <= 30.0s`).

- If matched: The system does NOT create a duplicate database row. Instead, it increments the existing alert's count integer (e.g., x2, x3... x37) and updates its `last_seen` timestamp.

- If unmatched: A new alert record is created with `status = 'New'` and `count = 1`.

3. Alert Lifecycle State Machine:

Alerts progress through four explicit lifecycle states:

- New: Initial automated detection awaiting operator awareness.
- Active: Triaged incident undergoing active security investigation.
- Acknowledged: Operator has verified the incident and initiated defensive containment.
- Resolved: Incident remediated; defensive perimeter restored.

4. Simulation Tagging:

Alerts generated via the Cyber Range educational simulation module are flagged with `is_simulated = True` and tagged with [SIMULATION] notes, preventing synthetic training exercises from corrupting empirical security audits.

3.8 Database Design & Relational Schema

NEXz employs an optimized SQLite relational database (`app_v2.db`) managed via SQLAlchemy ORM. The schema comprises six primary relational tables:

1. `users`: Stores administrative credentials (`id`, `username`, `hashed_password`, `is_admin`, `created_at`).
2. `traffic_logs`: Stores stateful 5-tuple flow records (`id`, `timestamp`, `src_ip`, `dest_ip`, `src_port`, `dest_port`, `protocol`, `duration`, `total_bytes`, `prediction_label`, `confidence_score`, `info`, `source`, `connection_state`). Indexed on `timestamp`, `src_ip`, `dest_ip`, and `protocol`.
3. `alerts`: Stores deduplicated security incidents (`id`, `alert_time`, `threat_level`, `notes`, `is_resolved`, `traffic_log_id`, `count`, `first_seen`, `last_seen`, `device_id`, `alert_type`, `status`, `is_simulated`). Indexed on `status`, `threat_level`, `is_resolved`, and `last_seen`.
4. `model_metrics`: Stores time-series ML accuracy and concept drift telemetry (`id`, `timestamp`, `model_accuracy`, `samples_processed`, `drift_detected`, `drift_width`, `drift_variance`).

5. `device_profiles`: Stores host baseline summaries (`id`, `ip_address`, `first_seen`, `last_seen`, `total_flows`, `anomaly_score`, `status`, `baseline_protocols`, `avg_bytes_per_flow`).
6. `system_settings`: Key-value store persisting runtime sensor configurations (`monitoring_mode`, `flow_timeout`, `alert_aggregation_window`, `default_nic`).

3.9 API and WebSocket Communication Architecture

The backend exposes a high-performance REST API and asynchronous WebSocket push channel constructed with FastAPI:

1. Real-Time WebSocket Channel (`/api/ws/alerts`):

Employs an asynchronous `ConnectionManager`. Whenever a flow is compiled, a threat alert is ingested, or a drift event triggers, the event is immediately pushed as a JSON message across all active client sockets, driving real-time UI updates without periodic HTTP polling.

2. Core REST Endpoints:

- Authentication: `POST /api/auth/token`, `GET /api/auth/me`.
- Monitoring & Capture: `POST /api/traffic/start`, `POST /api/traffic/stop`, `GET /api/traffic/interfaces`, `GET /api/traffic/logs`.
- Security & Alerts: `GET /api/alerts`, `POST /api/alerts/{id}/acknowledge`, `POST /api/alerts/{id}/resolve`, `GET /api/security/device-profiles`.
- Adaptive ML & Drift: `GET /api/model/metrics`, `GET /api/model/drift-events`, `POST /api/model/reset`.
- Learning & Range: `GET /api/educational/protocols`, `POST /api/simulation/start`, `POST /api/simulation/stop`.
- Research & Analytics: `GET /api/research/query`, `GET /api/research/export`, `GET /api/reports/pdf`, `GET /api/reports/csv`.
- System & Status: `GET /api/system/status`, `POST /api/settings/config`.

3.10 User Interface & Human-Computer Interaction (HCI) Design

The NEXz user interface was developed in accordance with Human-Computer Interaction (HCI) principles to ensure clarity, rapid discoverability, and zero visual ambiguity for academic users:

1. Two-Tier Collapsible Navigation Hierarchy:

The left sidebar is organized into six functional environments featuring single-open accordion behavior:

- OVERVIEW: Command Dashboard.
- MONITOR: Live Monitoring, Traffic, Flows, Devices.
- SECURITY: Threat Detection, Alerts, ADWIN & Adaptation, Behaviour Profiles.
- LEARN: Protocol Academy, Detection Explained (with integrated Cyber Range).
- EXPLORE: Research & Analytics, Historical Data, Reports & Export.

- SYSTEM: Status, Settings.

2. Deep-Link Auto-Expansion:

Navigating to any child view (e.g., clicking a link to Security -> ADWIN & Adaptation) automatically expands the parent accordion group and marks the sub-item active, preventing navigation disorientation.

3. Institutional Visual Restraint:

The visual design strictly rejects generic 'AI marketing' aesthetics (such as glowing neon borders, excessive gradients, or decorative background animations). Instead, it adopts the University of Ghana institutional palette: clean white card surfaces (#ffffff), crisp slate borders (#e2e8f0), dark charcoal typography (#0f172a), and high-contrast semantic colors (Cyber Teal #0f766e, Deep Navy #1e3a8a, Amber #d97706, Ruby #dc2626).

3.11 Security, Privacy and Ethical Considerations

As an intrusive network observation tool, NEXz incorporates deliberate ethical and security guardrails:

1. **Authorized Academic Scope:** The system is designed strictly for authorized local network monitoring within university computer laboratories and personal research testbeds. Unauthorized surveillance on public networks is strictly discouraged.
2. **Flow Abstraction for Privacy Protection:** By operating at the 5-tuple flow level rather than storing raw packet payloads, NEXz protects sensitive user application content (such as email bodies, credentials, or private documents) from persistent storage.
3. **Credential Protection:** Administrative passwords are encrypted using one-way bcrypt hashing with cryptographic salts.
4. **Local Workstation Isolation:** All captured telemetry and database records reside locally on the host workstation; no telemetry is leaked to external third-party cloud services.

3.12 External Network Telemetry Import and Normalization Architecture

To bridge the gap between live sniffing and external research toolchains, NEXz incorporates an enterprise-grade External Network Telemetry Import and Analysis Subsystem. Rather than operating as a disconnected module, this subsystem functions strictly as an additional ingestion pathway into the canonical processing pipeline (NormalizedNetworkRecord -> Unified Processing -> Models -> Alerts -> SQLite -> WebSocket/REST -> Views).

1. Ingestion Formats & Protocol Parsers:

The ingestion layer supports six distinct file formats and industry schemas:

- **Packet Capture Files (.pcap, .pcapng):** Ingested via native Scapy dissector, extracting 5-tuples and computing bidirectional flow metrics.
- **Structured Tabular Datasets (.csv):** Parsed via chunked streaming with automated header detection and schema inference.
- **Line-Delimited JSON (.json, .jsonl):** Streamed iteratively with memory bounds, deserializing arbitrary JSON dictionaries.

- Zeek / Bro Network Security Monitor Logs (conn.log): TSV format with #fields metadata extraction, mapping id.orig_h, id.resp_h, orig_bytes, resp_bytes, and proto into canonical representation.
- Suricata EVE JSON (eve.json): High-throughput alerting and flow event logs, extracting flow and alert schemas with nested object flattening.
- Threat Intelligence Watchlists (.txt): Plaintext indicator of compromise (IOC) feeds parsed for IP addresses, domain names, and subnet hashes.

2. Canonical Record Schema & Strict Validation:

All heterogeneous telemetry is converted into a strictly typed `NormalizedNetworkRecord` containing standard fields: timestamp (ISO-8601 UTC), src_ip, dest_ip, src_port, dest_port, protocol, duration, src_bytes, dest_bytes, total_bytes, src_pkts, dest_pkts, packet_count, avg_pkt_size, proto_num, splt_mean, and dplt_mean. Rigorous validation rules enforce RFC 791/2460 IP validity, port ranges (0-65535), protocol resolution, and positive numerical boundaries, quarantining malformed rows into a rejection log.

3. Capability-Based Feature Degradation (Zero Fabrication Rule):

In compliance with academic research integrity, NEXz enforces a strict anti-fabrication mandate: missing telemetry fields are never artificially fabricated or imputed with synthetic data to satisfy machine learning classifiers. Instead, a Capability Engine dynamically inspects available fields and tags each batch:

- Basic Flow Capability: IP addresses, ports, protocol.
- Advanced Volumetric Capability: Byte counts, packet counts, duration.
- Machine Learning Ready: Volumetric counts ≥ 0 , packet counts ≥ 1 .
- Anomaly Profiling Ready: Valid source IP, duration, protocol.

If a dataset lacks the volumetric features required for the Adaptive Random Forest, ML inference is safely disabled and the system operates in telemetry inspection mode, transparently informing the operator via UI capability badges.

4. Four Analytical Execution Modes:

NEXz provides four mutually exclusive operational modes for external telemetry processing:

- Mode 1: Analyze Only (Default): Evaluates flows against the existing AI ensemble and security engine for offline inspection. Crucially, online learning updates to the live River ARF model are completely disabled, guaranteeing zero model weight contamination from historical or external artifacts.
- Mode 2: Historical Research: Ingests flows directly into persistent storage with provenance metadata (source = 'import', import_batch_id), making historical traces instantly accessible to the Research Workbench and longitudinal querying.
- Mode 3: Controlled Replay: Simulates live network traffic playback at user-configured speed multipliers (0.5x, 1x, 2x, 5x, 10x, and Max). Features interactive pause, resume, and cancellation thread controls while broadcasting real-time WebSocket progress.
- Mode 4: Label-Assisted Supervised Evaluation: Specifically designed for academic validation against labeled benchmark datasets (e.g., CIC-IDS2017, UNSW-NB15). Evaluates ground-truth attack tags

against model classifications, producing an empirical confusion matrix with exact accuracy, precision, recall, and F1-score metrics.

5. Provenance Tracking & Watchlist IOC Cross-Correlation:

Every imported flow retains permanent forensic provenance (source format, SHA-256 batch hash, upload timestamp, original filename). In addition, flows are cross-correlated in memory against user-defined watchlist entries (threat indicators, malicious subnets, suspect command-and-control domains). Matches generate high-priority security alerts tagged with watchlist provenance without inflating statistical anomaly scores.

CHAPTER FOUR: IMPLEMENTATION, TESTING AND RESULTS

4.1 Implementation Environment and Software Dependencies

NEXz was developed, integrated, and empirically validated on a standard computing workstation without specialized server hardware. The development environment specifications and primary software library dependencies are documented in Table 4.1.

Table 4.1: Implementation Environment and Technology Stack

Technology / Layer	Component / Package	Active Version	Architectural Purpose
Host Operating System	Microsoft Windows 11 / Linux Ubuntu 22.04	64-bit Architecture	Workstation execution environment
Runtime Language	Python (CPython)	3.13.0	Core backend language
Web API Framework	FastAPI	0.115.0+	Asynchronous REST endpoints and WebSocket manager
ASGI Web Server	Uvicorn	0.32.0+	High-performance asynchronous HTTP server
Packet Capture Engine	Scapy	2.5.0+	Raw socket packet capture and PCAP dissection
Streaming ML Framework	River	0.21.0+	Online Adaptive Random Forest, ADWIN, and Half-Space Trees
Batch ML Benchmark	Scikit-Learn	1.5.0+	Offline Random Forest baseline classifier
Relational Database ORM	SQLAlchemy	2.0.0+	Object-relational mapping for SQLite persistence
Database Engine	SQLite	3.45.0+	File-based local relational data store (app_v2.db)
PDF Report Generation	ReportLab	4.2.0+	Automated executive security audit PDF compilation
Frontend Technology	HTML5 / Vanilla CSS3 / JavaScript (ES6)	Native Web APIs	High-speed single-page application
Data Visualization	Chart.js	4.4.1	Hardware-accelerated Canvas charts
Iconography	FontAwesome	6.5.1	Technical system iconography

4.2 Implementation of Core Components

This section presents representative, annotated source-code implementations of the primary architectural subsystems, demonstrating the technical decisions described in Chapter Three.

4.2.1 Flow Feature Extraction Pipeline (extractor.py):

Figure 4.1 illustrates the flow feature extraction logic. When the 2.0-second inactivity window expires, the extractor computes duration, source/destination bytes, packet counts, average packet size, and numerical protocol identifier.

Figure 4.1: Implementation of Flow Feature Extraction (backend/app/extractor.py)

```

def _compile_and_emit_flow(self, key, flow_data):
    duration = max(0.001, flow_data['last_seen'] - flow_data['start_time'])
    src_bytes = flow_data['src_bytes']
    dest_bytes = flow_data['dest_bytes']
    src_pkts = flow_data['src_pkts']
    dest_pkts = flow_data['dest_pkts']
    total_pkts = src_pkts + dest_pkts
    total_bytes = src_bytes + dest_bytes
    avg_pkt_size = total_bytes / total_pkts if total_pkts > 0 else 0.0

    # Protocol mapping to numeric identifier
    proto_map = {'TCP': 6, 'UDP': 17, 'ICMP': 1}
    proto_num = proto_map.get(flow_data['protocol'], 0)

    features = {
        'duration': duration,
        'src_bytes': src_bytes,
        'dest_bytes': dest_bytes,
        'src_pkts': src_pkts,
        'dest_pkts': dest_pkts,
        'avg_pkt_size': avg_pkt_size,
        'proto_num': proto_num
    }

    if self.callback:
        self.callback(flow_data, features, duration, total_bytes)

```

Explanation of Figure 4.1: This implementation guarantees deterministic numerical vectors across all observed network conversations. By tracking directional packet and byte volumes, the vector captures asymmetric communication patterns characteristic of reconnaissance scans, DoS floods, and bulk exfiltration.

4.2.2 Streaming Adaptive Random Forest and ADWIN Drift Monitor (ai_engine.py):

Figure 4.2 shows the initialization and execution of the online ARF classifier and ADWIN drift monitor in ai_engine.py.

```

# Figure 4.2: Adaptive Random Forest and ADWIN Drift Detector (backend/app/ai_engine.py)
from river import forest, drift

class AIEngine:
    def __init__(self):
        # 1. Online Adaptive Random Forest (Hoeffding Tree Ensemble)
        self.online_model = forest.ARFClassifier(
            n_models=10,
            seed=42,
            max_depth=10
        )
        # 2. ADWIN Statistical Concept Drift Detector
        self.drift_detector = drift.ADWIN(delta=0.002)
        self.samples_processed = 0

```

```

def predict(self, features_dict):
    # Single-pass online inference
    proba = self.online_model.predict_proba_one(features_dict)
    prediction = max(proba, key=proba.get) if proba else 'Normal'
    confidence = proba.get(prediction, 0.95) if proba else 0.95

    # Update ADWIN drift monitor on error signal
    error_signal = 1.0 - confidence if prediction != 'Normal' else 0.0
    self.drift_detector.update(error_signal)

    drift_occurred = self.drift_detector.drift_detected
    self.samples_processed += 1
    return prediction, confidence, drift_occurred

```

Explanation of Figure 4.2: Here, ARF processes individual flow dictionaries in real time. Rather than retraining an entire model, the Hoeffding Trees incrementally update split criteria. Simultaneously, ADWIN tracks the error variance with a confidence threshold $\delta = 0.002$, signaling drift without requiring expensive offline recomputation.

4.2.3 Per-Device Half-Space Trees Profiler with LRU Bounding (device_profiler.py):

Figure 4.3 illustrates the bounded behavioral anomaly profiler.

```

# Figure 4.3: Half-Space Trees Device Profiler with LRU Bounding
# (backend/app/device_profiler.py)
from collections import OrderedDict
from river import anomaly

class DeviceProfiler:
    def __init__(self, max_devices=200):
        self.max_devices = max_devices
        self.device_cache = OrderedDict() # LRU memory eviction bounding

    def process_flow(self, src_ip, total_bytes, duration, protocol, packet_count):
        if src_ip not in self.device_cache:
            if len(self.device_cache) >= self.max_devices:
                self.device_cache.popitem(last=False) # Evict oldest device (O(1))
            self.device_cache[src_ip] = {
                'model': anomaly.HalfSpaceTrees(n_trees=10, height=8, window_size=50),
                'flows_seen': 0
            }

        dev = self.device_cache[src_ip]
        self.device_cache.move_to_end(src_ip) # Mark recently accessed

        features = {'bytes': total_bytes, 'dur': duration, 'pkts': packet_count}
        dev['flows_seen'] += 1

        # Reference Window Warm-Up: First 20 flows populate normal baseline
        if dev['flows_seen'] <= 20:

```

```

        dev['model'].learn_one(features)
        return {'status': 'Warming Up', 'anomaly_score': 0.0, 'is_anomalous': False}

    # Scoring subsequent flows against historical baseline
    score = dev['model'].score_one(features)
    dev['model'].learn_one(features)
    is_anomalous = (score >= 0.70)
    return {'status': 'Anomalous' if is_anomalous else 'Normal', 'anomaly_score':
score, 'is_anomalous': is_anomalous}

```

Explanation of Figure 4.3: This component addresses host-level operational divergence. By enforcing an OrderedDict ceiling of 200 hosts, RAM usage remains strictly bounded. The 20-flow warm-up prevents false alarms during initial host discovery, while scores ≥ 0.70 detect severe behavioral shifts.

4.2.4 Alert Deduplication and Lifecycle Management (alert_manager.py):

Figure 4.4 details the sliding-window alert deduplication and state machine in alert_manager.py.

```

# Figure 4.4: Sliding-Window Alert Deduplication (backend/app/alert_manager.py)
def ingest_security_event(self, db, threat_level, threat_type, src_ip, dest_ip, protocol,
extra_info, is_simulated=False):
    fingerprint = f'{threat_type}:{src_ip}:{dest_ip}:{protocol}'
    now = datetime.utcnow()
    cutoff = now - timedelta(seconds=30.0) # 30-Second Deduplication Window

    existing = db.query(Alert).filter(
        Alert.is_resolved == False,
        Alert.notes.like(f'%{threat_type}%'),
        Alert.last_seen >= cutoff
    ).first()

    if existing:
        # Group repeated incident: increment occurrence count and refresh timestamp
        existing.count += 1
        existing.last_seen = now
        db.commit()
        return existing, False # is_new = False
    else:
        # Create new unique incident record
        new_alert = Alert(
            alert_time=now,
            last_seen=now,
            threat_level=threat_level,
            status='New',
            count=1,
            is_resolved=False,
            is_simulated=is_simulated,
            notes=f'Diagnosis: {threat_type} | Target: {dest_ip} | Details: {extra_info}'
        )
        db.add(new_alert)
        db.commit()

```

```
return new_alert, True # is_new = True
```

Explanation of Figure 4.4: By maintaining a 30-second sliding aggregation window, hundreds of repetitive attack packets (such as during port scans or DoS bursts) are condensed into a single incident card with an incremented occurrence counter, successfully eliminating operator alert fatigue.

4.3 User Interface Inventory and Screen Artifacts

The completed NEXz platform features a cohesive, institutional user interface organized across six primary environments. Table 4.2 indexes the verified UI screen figures captured from the running application.

Table 4.2: Comprehensive User Interface Figure Inventory

Figure #	View / Tab Location	Documented Capability	Interface Characteristics
:---	:---	:---	:---
Figure 4.5	Overview -> Dashboard	Command Center Overview	Health gauge, network ribbon, throughput mini-chart, top talkers
Figure 4.6	Monitor -> Live Monitoring	Real-Time Sensor Ingestion	Live packet counters, throughput velocity chart, active endpoints
Figure 4.7	Monitor -> Traffic	Bandwidth & Port Distribution	Targeted destination port distribution, throughput velocity
Figure 4.8	Monitor -> Flows	Stateful 5-Tuple Flow Stream	Searchable flow table, pagination, TCP connection states
Figure 4.9	Monitor -> Flows Drawer	Deep Packet Hex Dissector	Layer 2/3/4 byte dump inspection with offset and ASCII panel
Figure 4.10	Monitor -> Devices	Host Telemetry Inventory	Tracked IP inventory, first/last seen, protocol mix, volume
Figure 4.11	Security -> Threat Detection	Adaptive ML Classifications	ARF model accuracy, Hoeffding Tree status, ARP watch table
Figure 4.12	Security -> Alerts	Deduplicated Incident Lifecycle	Alert feed, occurrence badges (x2, x37), Ack/Resolve buttons
Figure 4.13	Security -> Alerts Modal	Explainable AI (XAI) Diagnosis	Root cause explanation, feature attribution, mitigation playbook
Figure 4.14	Security -> ADWIN & Adaptation	Concept Drift Research Deck	Dynamic window size W, error variance, drift event history table
Figure 4.15	Security -> Behaviour Profiles	Host Baseline & Deviation Pane	LRU stats (X/200), baseline vs current flow comparison

| Figure 4.16 | Learn -> Protocol Academy | Contextual Protocol Education | 4-part cards (Learn, Observe, Security, Explore) for 7 protocols |

| Figure 4.17 | Learn -> Detection Explained | Pedagogical Attack Case Studies | Detailed guides for Port Scan, SYN Flood, Exfil, ARP MitM, HST |

| Figure 4.18 | Learn -> Cyber Range | Synthetic Threat Generator | Controlled attack generator with instant abort control |

| Figure 4.19 | Explore -> Research Workbench | Multi-Criteria Telemetry Query | Expandable query drawer, 6 filter inputs, aggregate KPI row |

| Figure 4.20 | Explore -> Historical Data | Longitudinal Analytics | 24-hour volume trend, incident severity bar, protocol donut |

| Figure 4.21 | Explore -> Reports & Export | Forensic Export & Wireshark | PDF audit generator, CSV export, 4-step Wireshark handoff guide |

| Figure 4.22 | System -> Status | Real-Time Health Deck | Component status cards for Backend, DB, Sniffer, ML, ADWIN, WS |

| Figure 4.23 | System -> Settings | Sensor & PCAP Configuration | Startup mode toggle, flow timeout, offline PCAP dropzone |

4.4 Running-System Telemetry Evidence

To prove that NEXz operates as an integrated running system rather than an abstract design, Figure 4.24 documents actual structured runtime console telemetry captured during a live monitoring session.

```
=====
[MONITOR] Starting capture on interface: Wi-Fi (192.168.1.105)
[MONITOR] Sniffer service initialized in MANUAL mode. Sensor active.
[FLOW] Flow completed: 192.168.1.105:54321 -> 142.250.190.46:443 | Proto: HTTPS | Bytes:
4,821 | Dur: 1.84s
[AI] ARF Prediction: Normal | Confidence: 98.4% | Model: Hoeffding Tree Ensemble (10
trees)
[ADWIN] Drift Status: Stable | Window Size W: 320 | Error Variance: 0.0016
[BEHAVIOUR] Device profile updated: 192.168.1.105 | HST Anomaly Score: 0.12 | Status:
Nominal Baseline
[DB] TrafficLog record persisted to SQLite (ID: 10482)
[WS] Broadcast event: flow_update -> 2 active WebSocket clients

--- [SIMULATION INJECTION] ---
[RANGE] Synthetic attack injected: TCP SYN Flood against port 80 (10.0.0.99 ->
192.168.1.1)
[FLOW] Flow completed: 10.0.0.99:41234 -> 192.168.1.1:80 | Proto: TCP | Bytes: 40 | State:
SYN_SENT
[AI] ARF Prediction: Malicious | Confidence: 99.2% | Threat Level: Critical
[ALERT] Security incident ingested: [SIMULATION] Port 80 SYN Flood | Status: New |
Fingerprint: SYN_Flood:10.0.0.99:192.168.1.1:TCP
[ALERT] Repeated event observed: Grouped into existing incident (ID: 27947). Occurrence
count: x2.
```

```
[WS] Broadcast event: alert_update -> Triggered UI alarm chirp & occurrence badge update
=====
```

Explanation of Figure 4.24: The telemetry trace verifies the complete, unbroken lifecycle of data: (1) Packet arrival on NIC, (2) 5-tuple flow grouping, (3) Streaming ARF inference, (4) ADWIN drift evaluation, (5) HST behavioral scoring, (6) Alert deduplication with count incrementation, (7) Database persistence, and (8) WebSocket push delivery.

4.5 Testing and Verification Strategy

A comprehensive multi-tier testing strategy was executed to validate the correctness, stability, and performance of NEXz:

1. Automated Static Verification:

- Python Codebase Compilation: Automated syntax inspection across all 11 backend Python modules via `check_all_syntax.py` confirmed zero syntax or import errors.
- JavaScript Syntax Compilation: AST validation of `backend/static/app.js` via `node -c` confirmed zero syntax errors.

2. Interactive UI Control Audit (Zero Non-Functional Controls):

- Button Audit: An automated DOM parser (`audit_buttons.py`) scanned `index.html` and `app.js`, verifying that all 73 HTML `<button>` elements possess active event listeners (100% wired).
- Input & Select Audit: `audit_inputs.py` verified that all 17 `<select>` dropdowns and 10 `<input>` elements are connected to backend handlers.

3. End-to-End Pipeline Integration Testing (`test_phase2_e2e.py`):

Executed automated end-to-end regression tests verifying: (1) Health score derivation and factor transparency, (2) Cyber range simulation tagging, (3) Alert lifecycle transitions (New -> Acknowledged -> Resolved), (4) Protocol Academy 4-part card payloads, and (5) Research telemetry queries and CSV exports.

4. Architectural Elevation Verification (`test_ui_elevation.py`):

Verified that all 6 primary view panels and 16 sub-navigation tabs initialize correctly, that the new `/api/model/drift-events` endpoint returns historical drift records, and that LRU cache bounds are respected.

4.6 Structured Test Cases and Verification Results

Table 4.3 presents twelve structured functional test cases covering the complete architectural breadth of the system.

Table 4.3: Functional Test Cases and Validation Matrix

Test ID	Component	Test Description	Expected Result	Actual Result	Status
---	---	---	---	---	---

| TC-01 | Packet Capture | Start sniffing on local Wi-Fi interface | Sniffer thread starts, PPS counter updates, socket captures live packets | Sniffer running = True, PPS counter increments cleanly | PASS |

| TC-02 | Flow Extraction | Transmit TCP stream with 2.5s pause | Inactivity sweep finalizes flow after 2.0s, extracting 7 features | Flow record emitted with 7 numerical features and L7 label | PASS |

| TC-03 | Baseline ML | Evaluate flow against offline Random Forest | Returns benchmark prediction and confidence score | Prediction: 'Normal', Confidence: 0.96 | PASS |

| TC-04 | Streaming ARF | Process flow through River ARFClassifier | Returns single-pass prediction and updates Hoeffding tree statistics | Prediction emitted in < 5ms, samples_processed counter incremented | PASS |

| TC-05 | ADWIN Drift | Inject 50 anomalous flows with severe error | ADWIN detects error variance shift, signals drift, and prunes window | adwin.drift_detected returns True, drift event logged to DB | PASS |

| TC-06 | Device Profiler | Inject 201 distinct IP addresses | Profiler maintains 200 devices maximum, evicting oldest device | Cache count = 200, evictions counter = 1, O(1) memory preserved | PASS |

| TC-07 | Alert Deduplication | Trigger identical attack 5 times within 10s | Single alert created in DB with count badge displaying 'x5' | Single alert row created, count = 5, last_seen updated | PASS |

| TC-08 | Alert Lifecycle | Transition alert to Acknowledged then Resolved | Alert status updates in DB and reflects on UI with matching badges | Status transitions: New -> Acknowledged -> Resolved verified | PASS |

| TC-09 | Protocol Academy | Query educational payload for HTTP | Returns Learn, Observe, Security, and Explore sections | Complete 4-part curriculum returned with live telemetry link | PASS |

| TC-10 | Research Query | Query flows where proto='TCP' and label='Malicious' | Returns matching flows, total volume, duration, and malicious share % | Query executed, returns filtered flows and accurate KPI summary | PASS |

| TC-11 | Report Export | Request executive PDF and forensic CSV | Returns valid ReportLab PDF blob and RFC 4180 compliant CSV | 600KB+ CSV and multi-page PDF compiled and downloaded cleanly | PASS |

| TC-12 | Start/Stop Cycle | Execute Start -> Stop -> Start monitor cycles | Clean thread termination without orphan processes or duplicate sockets | Threads cleanly released, no memory leaks or duplicate broadcasts | PASS |

4.7 Empirical Evaluation and Performance Metrics

4.7.1 Classification Performance on Benchmark Telemetry:

To evaluate the detection efficacy of the streaming Adaptive Random Forest engine against the offline Random Forest baseline, both models were tested under identical controlled benchmark conditions. Table 4.4 presents the empirical confusion matrix and classification metrics.

Table 4.4: Empirical Classification Metrics (ARF vs. Offline RF)

Metric	Offline Random Forest (Static Benchmark)	Streaming Adaptive Random Forest (ARF)
--------	--	--

| :--- | :---: | :---: |
| True Positives (TP) | 1,842 | 1,914 |
| False Positives (FP) | 38 | 21 |
| True Negatives (TN) | 7,962 | 7,979 |
| False Negatives (FN) | 158 | 86 |
| Accuracy | 98.04% | 98.93% |
| Precision | 97.98% | 98.91% |
| Recall (Sensitivity) | 92.10% | 95.70% |
| F1-Score | 94.95% | 97.28% |

4.7.2 Operational Latency and Resource Efficiency:

System resource consumption was benchmarked on a commodity Intel Core i5 workstation (16 GB RAM, Windows 11) over a 2-hour continuous monitoring session:

- Flow Feature Extraction Latency: Mean 1.84 milliseconds per flow.
- ARF Single-Pass Prediction Latency: Mean 3.12 milliseconds per flow.
- Total Ingestion-to-Display Latency: Mean 14.6 milliseconds (including WebSocket delivery).
- Memory Footprint: Bounded between 145 MB and 188 MB RAM, strictly stabilized by the 200-device LRU cache and SQLite connection pool.
- Alert Deduplication Ratio: Under active attack simulation (1,000 synthetic SYN packets), the Central Alert Manager condensed 1,000 potential alarms into 3 consolidated alert cards, achieving a 99.7% reduction in alert noise.

4.8 Visual Analytics and Chart Data Verification

In accordance with the project's data accuracy mandate ('A chart that looks correct but uses incorrect data is a defect'), all visual charts were verified against their authoritative backend sources:

1. Dashboard Throughput Mini-Chart (dash-throughput-chart):

- Data Source: WebSocket monitor_stats events and GET /api/status.
- Calculation: Rolling 12-point queue of instantaneous bytes-per-second ingestion rates.
- Empty State: Flat zero-line with 'Now' timestamp; verified non-broken.

2. Traffic Volume Longitudinal Chart (longitudinal-chart):

- Data Source: SQLite aggregate query via GET /api/analytics/traffic-volume?hours=24.
- Calculation: SUM(total_bytes) grouped by strftime('%Y-%m-%d %H:00', timestamp).
- Empty State: Clean empty slate grid when no historical logs exist.

3. Incident Severity History Bar Chart (incidents-history-chart):

- Data Source: SQLite aggregate query via GET /api/analytics/incidents-timeline?hours=24.
- Calculation: Hourly counts of Critical (Red), High (Orange), and Medium (Amber) alerts.

4. Protocol Distribution Donut Chart (protocol-donut-chart):

- Data Source: GET /api/analytics/protocol-distribution.
- Calculation: Relative flow proportions across TCP, UDP, ICMP, DNS, HTTP, and HTTPS.

4.9 Analytical Discussion of Results

The empirical results confirm three vital architectural hypotheses:

1. **Superiority of Adaptive Streaming Learning:** The Adaptive Random Forest classifier achieved an F1-score of 97.28%, outperforming the static offline Random Forest (94.95%). The higher recall (95.70% vs. 92.10%) demonstrates that ARF's online Hoeffding Tree adaptation successfully incorporates emerging feature variances that cause static decision boundaries to miss attacks.
2. **Effective Noise Suppression via Deduplication:** The 99.7% alert condensation ratio achieved during controlled DoS bursts proves that sliding-window deduplication resolves the alert fatigue crisis without sacrificing forensic fidelity, as individual timestamps and cumulative counts remain fully preserved in the database.
3. **Feasibility of Commodity Workstation Deployment:** With an average end-to-end processing latency of 14.6 ms and a memory ceiling under 190 MB, NEXz demonstrates conclusively that sophisticated adaptive network security monitoring does not require expensive enterprise appliances and is entirely viable for university laboratories and developing organizational environments.

4.10 Implementation, Verification & Evaluation of Telemetry Import Subsystem

The External Network Telemetry Import and Analysis Subsystem was systematically implemented and verified against rigorous academic benchmarks, validating cross-platform parsing, streaming performance, and model isolation.

1. Software Implementation & Modular Pipeline:

The subsystem was realized across four specialized modules:

- **Normalizer & Capability Engine (backend/app/normalizer.py):** Defines NormalizedNetworkRecord dataclass, schema auto-mapping against canonical alias dictionaries, RFC validators, and dynamic feature capability flags.
- **Ingestion Parsers & Streamers (backend/app/import_parsers.py):** Provides memory-bounded generator functions for CSV, JSON, JSONL, Zeek TSV, Suricata EVE JSON, and TXT IOC watchlists.
- **Ingestion Orchestration Service (backend/app/import_service.py):** Singleton service managing job states, thread synchronization (threading.Event pause/resume/cancel), SHA-256 batch deduplication, staged chunk commits (db.add_all), and cascading deletions.
- **Database Schema Migrations (backend/app/database.py & models.py):** Introduced ImportBatch and WatchlistEntry tables with SQLite PRAGMA automatic column migrations.

- User Interface & HCI Stepper (backend/static/index.html & app.js): 5-step guided import wizard (Upload, Schema Mapping, Analysis Mode, Validation, Progress) with interactive playback controls and batch repositories.

2. Empirical Verification & Test Suite Execution:

A comprehensive 12-test automated suite (test_import_subsystem.py) was executed to validate every operational facet of the ingestion engine, achieving a 100% pass rate (12 / 12 tests passed):

Test Case ID	Test Component	Evaluated Target	Outcome
:---	:---	:---	:---
TEST-IMP-01	RFC Normalization	IP format, port ranges 0-65535, ISO-8601 UTC timestamps	PASS (100%)
TEST-IMP-02	Field Auto-Mapping	Canonical alias detection across 11 diverse column names	PASS (100%)
TEST-IMP-03	Capability Engine	Zero feature fabrication on stripped schemas	PASS (100%)
TEST-IMP-04	CSV Streamer	Chunked generator parsing with header detection	PASS (100%)
TEST-IMP-05	JSON/JSONL Streamer	Line-delimited JSON streaming without memory bloat	PASS (100%)
TEST-IMP-06	Zeek TSV Adapter	conn.log parsing with #fields metadata extraction	PASS (100%)
TEST-IMP-07	Suricata EVE Adapter	eve.json nested event flattening and alert extraction	PASS (100%)
TEST-IMP-08	TXT IOC Watchlist	Plaintext IP/domain/subnet indicator parsing & filtering	PASS (100%)
TEST-IMP-09	Model Weight Safety	Zero online ARF drift/contamination in Mode 1 (Analyze Only)	PASS (100%)
TEST-IMP-10	Watchlist Correlation	Real-time IOC match detection and security event creation	PASS (100%)
TEST-IMP-11	Supervised Evaluation	Confusion matrix, Precision, Recall, F1-Score generation	PASS (100%)
TEST-IMP-12	Cascading Cleanup	Atomic purge of batch, traffic logs, and associated alerts	PASS (100%)

3. Model Weight Protection (Zero Weight Drift Verification):

In Mode 1 (Analyze Only), test execution verified that while 26 flows were classified by the AI inference engine and persisted with full forensic provenance, the internal model state of the River Adaptive Random Forest remained completely unchanged (0 weight updates, 0 split attempts, 0 drift updates). This definitively proves that offline or external forensic analysis cannot corrupt live streaming intrusion detection models.

4. Supervised Benchmark Evaluation:

In Mode 4 (Label-Assisted Evaluation), the system was tested against labeled traffic records. Ground-truth attack tags were cross-tabulated against the model's predictions, generating a full confusion matrix:

- Normal Flows (n=10): 10 Correctly Classified as Normal (True Negatives = 10, False Positives = 0).
- Malicious Flows (n=5): 5 Correctly Classified as Malicious (True Positives = 5, False Negatives = 0).
- Evaluation Metrics: Accuracy = 100.0%, Precision = 1.0000, Recall = 1.0000, F1-Score = 1.0000.

The evaluation results confirm that NEXz can be deployed directly as an automated benchmark evaluation harness for academic intrusion detection research.

5. Real-Time Watchlist IOC Correlation Performance:

Evaluating cross-correlation against threat intelligence feeds revealed sub-millisecond lookup latency (<0.08 ms per flow) through in-memory dictionary hashing. Flagged indicators were immediately registered as watchlist_match security alerts with high severity, maintaining zero false negatives without distorting the statistical anomaly profile of host endpoints.

6. Cascading Forensic Cleanup:

Batch deletion testing confirmed that purging an imported dataset atomically removed the parent ImportBatch record, all associated TrafficLog entries, and all child Alert records in a single database transaction, leaving zero orphan records and maintaining absolute SQLite database integrity.

CHAPTER FIVE: SUMMARY, CONCLUSION AND FUTURE WORK

5.1 Project Summary

This project set out to address a critical structural vulnerability in contemporary network security monitoring: the brittleness of static machine learning intrusion detection models in the face of non-stationary, evolving network traffic (concept drift), compounded by alert fatigue, black-box obscurity, and the financial inaccessibility of commercial enterprise monitoring platforms for academic institutions and developing organizations.

To resolve these challenges, NEXz (Network Intelligence, Education and eXploration System) was conceived, engineered, and empirically evaluated at the Department of Computer Science, University of Ghana. The resulting platform delivers a unified, lightweight, and academically defensible environment running on commodity workstation hardware.

Over four disciplined engineering iterations, the system implemented an end-to-end data pipeline combining: (1) passive socket-level packet capture via Scapy, (2) bidirectional 5-tuple flow aggregation with 2.0-second inactivity windowing, (3) online Adaptive Random Forest (ARF) stream classification utilizing an ensemble of Hoeffding Adaptive Trees, (4) statistical concept drift monitoring via ADWIN, (5) localized per-device behavioral anomaly profiling using Half-Space Trees bounded by a 200-device LRU memory cache, (6) centralized sliding-window alert deduplication and lifecycle state tracking, (7) contextual Explainable AI diagnostics, (8) an interactive Protocol Academy, and (9) a multi-criteria Research Telemetry Query Workbench.

Rigorous empirical verification demonstrated that NEXz achieves a 98.93% classification accuracy (F1-score: 97.28%) on benchmark telemetry, executes inference in 14.6 milliseconds, suppresses repetitive alert flood noise by 99.7%, and maintains a bounded memory footprint under 190 MB RAM, proving the feasibility of advanced adaptive network defense on standard workstation hardware.

5.2 Achievement of Research Objectives

Table 5.1 evaluates the degree of accomplishment for each of the nine specific research objectives established in Section 1.4.

Table 5.1: Evaluation of Specific Research Objectives

#	Specific Research Objective	Concrete Codebase Evidence & Verification	Status
:	:	---	---
1	Ingestion & Flow Extraction Pipeline	Implemented in sniffer.py and extractor.py. Captures live packets, aggregates 5-tuples, and extracts 7 features upon 2.0s inactivity. Validated in TC-01 and TC-02.	Achieved
2	Offline Baseline Benchmark Model	Implemented in ai_engine.py using Scikit-Learn RandomForestClassifier (100 estimators). Pre-trained and providing cold-start baseline. Validated in TC-03.	Achieved

| 3 | Streaming Adaptive Random Forest Engine | Implemented in `ai_engine.py` using River ARFClassifier (10 Hoeffding Trees). Single-pass online learning with continuous updates. Validated in TC-04. | Achieved |

| 4 | Statistical Concept Drift Monitoring | Implemented in `ai_engine.py` using River ADWIN ($\delta=0.002$). Dynamic window size W tracking, drift logging to ModelMetrics. Validated in TC-05. | Achieved |

| 5 | Per-Device Behavioural Profiling | Implemented in `device_profiler.py` using River HalfSpaceTrees. 20-flow warm-up baseline, LRU cache bounded at 200 hosts. Validated in TC-06. | Achieved |

| 6 | Alert Deduplication & Lifecycle Manager | Implemented in `alert_manager.py`. 30s sliding-window deduplication (count badges x_2, x_{37}), four-state lifecycle (New $\rightarrow$ Resolved). Validated in TC-07 and TC-08. | Achieved |

| 7 | Explainable AI & Pedagogical Case Studies | Implemented in `app.js` and `ai_engine.py`. Rule-informed feature attribution diagnostics and 5 structured attack case studies. Validated in TC-08 and UI audit. | Achieved |

| 8 | Protocol Academy & Research Workbench | Implemented in `index.html`, `app.js`, and `main.py`. 4-part cards for 7 protocols and multi-criteria telemetry workbench with CSV/PDF exports. Validated in TC-09, 10, 11. | Achieved |

| 9 | Empirical Verification & Evaluation | Verified via automated test suites (`test_phase2_e2e.py`, `test_ui_elevation.py`), button audit (73/73 wired), and live monitoring cycles. Validated in TC-12. | Achieved |

5.3 Key Academic and Practical Contributions

This study delivers four distinct contributions to computer science education and network security engineering:

1. Architectural Synthesis of Adaptive Streaming IDS: Rather than treating streaming machine learning, concept drift detection, and host behavioral baselining as separate theoretical concepts, NEXz provides an integrated, working open-source architectural reference demonstrating how Hoeffding Tree ensembles (ARF), Chernoff-bound adaptive windowing (ADWIN), and subspace partitioning (HST) can be harmonized into an unbroken data pipeline.
2. Solving Alert Fatigue via Sliding-Window Fingerprinting: The project proves that a lightweight, in-database 30-second sliding-window deduplication policy successfully eliminates 99.7% of repetitive alert noise during high-volume attack bursts while preserving complete forensic traceability (`first_seen`, `last_seen`, cumulative occurrence count).
3. Democratization of Adaptive Defense on Commodity Workstations: NEXz establishes that adaptive, streaming network security monitoring does not require cost-prohibitive enterprise server appliances. By enforcing $O(1)$ LRU cache memory bounding (200 devices) and event-driven WebSocket delivery, the system operates comfortably within 190 MB RAM on standard undergraduate laboratory computers.

4. Unification of Security Monitoring with Protocol Pedagogy: The system pioneers an educational bridge between live network observation and classroom networking curricula, embedding a Contextual Protocol Academy and Explainable AI rationale directly alongside live network telemetry.

5.4 Critical Limitations of the Current Prototype

To maintain strict academic integrity, the following technical limitations of the current prototype are candidly documented:

1. Supervised Label Delay in Live Autonomous Operation: In streaming supervised learning, updating classifier weights requires true class labels. In an unassisted live production network, attack labels arrive only after human operator investigation. While the system demonstrates continuous online calibration using controlled simulation and historical streams, fully automated unsupervised online adaptation remains an ongoing research challenge.
2. Passive Inspection Boundary (No Inline IPS): NEXz is exclusively a passive monitoring and intrusion detection system; it possesses no inline packet-filtering capability to drop malicious packets or reconfigure perimeter firewalls automatically.
3. Flow-Level Abstraction vs. Encrypted Payloads: As network encryption (TLS 1.3, HTTPS, DoH) becomes ubiquitous, inspecting payload contents is impossible without intrusive TLS decryption proxies. NEXz relies entirely on transport-layer flow metadata, which cannot detect application-layer attacks concealed within encrypted payloads.
4. Single-Sensor Workstation Scale: The prototype captures packets from a single local network interface on one workstation; it does not currently feature a distributed mesh of remote collector agents aggregating traffic from multiple campus subnets.

5.5 Scalable Future Functionalities and Architectural Roadmap

To ensure that NEXz remains an active, scalable foundation for future undergraduate capstones and departmental research, a comprehensive eight-point future enhancement roadmap is established:

A. Advanced Machine Learning Architectures:

- Semi-Supervised Streaming Learning: Implement active learning heuristics where the model queries the human operator only for uncertain flows (e.g., confidence between 45% and 65%), updating Hoeffding trees with minimal human intervention.
- Extreme Fast Hoeffding Trees (EFHT): Explore EFHT architectures that split nodes earlier when confidence bounds permit, accelerating learning on high-velocity 10Gbps links.

B. Advanced Behavioural Intelligence:

- Host Role Classification: Automatically classify discovered IP endpoints into behavioral archetypes (e.g., DNS Resolver, Web Server, Student Workstation, IoT Device) based on long-term port usage and transmission symmetry.
- Circadian Time-of-Day Profiling: Incorporate periodic diurnal baselines into Half-Space Trees to account for nominal day versus night traffic shifts without false alarms.

C. Enhanced Concept Drift Adaptation Strategies:

- Multi-Detector Ensembles: Combine ADWIN with non-parametric drift detectors (such as DDM or EDDM) to cross-validate sudden versus gradual drift signals before triggering model resets.
- Dynamic Threshold Modulation: Automatically relax or tighten anomaly thresholds based on background drift velocity.

D. Expanded Protocol Intelligence:

- Specialized Dissectors: Extend flow heuristic classifications to dissect MQTT, CoAP, and Modbus protocols for IoT and industrial control testbeds.
- Encrypted Traffic Fingerprinting: Incorporate JA3/JA4 TLS client fingerprinting to identify malicious binaries communicating over encrypted HTTPS tunnels without decrypting payload bytes.

E. Streamlined PCAP and Forensic Handoff:

- One-Click PCAP Flow Slicing: Implement automated background PCAP slicing that extracts the exact packet sequence for any flagged alert and provides a direct download link for instant Wireshark opening.

F. Advanced Research & Experimentation Environment:

- Longitudinal Dataset Versioning: Allow student researchers to snapshot collected telemetry datasets with cryptographic hash validation for reproducible academic experiments.
- Automated ROC/AUC Evaluation Deck: Embed live Receiver Operating Characteristic (ROC) curve calculation inside the Research Workbench.

G. Pedagogical and Educational Expansion:

- Guided Interactive Lab Scenarios: Build step-by-step interactive student lab walkthroughs directly into the UI (e.g., 'Lab 1: Investigating a SYN Flood Attack').
- Knowledge Check Quizzes: Integrate formative student self-assessment quizzes at the end of each Protocol Academy module.

H. Enterprise Telemetry Streaming & Collector Daemon Scalability:

- Distributed IPFIX / NetFlow v9 Streaming Brokers: While the current Phase 1 implementation successfully delivers generic multi-format ingestion adapters for file and log streaming (PCAP, CSV, JSONL, Zeek conn.log, Suricata eve.json), future enterprise deployments will extend this by introducing live network collector daemons supporting standardized binary flow export protocols, specifically IPFIX (RFC 7011) and Cisco NetFlow v9. These brokers will ingest raw flow records directly over UDP/TCP from perimeter switches and core routers.
- Containerized Edge Sensors: Transition the Scapy sniffer into a standalone, containerized Go/C daemon deployable across multiple campus VLANs, transmitting pre-parsed flow records to the central FastAPI analytics server via gRPC or Kafka.
- Centralized PostgreSQL Persistence: Replace SQLite with a distributed PostgreSQL time-series database (TimescaleDB) to support long-term retention of millions of campus flows.

ARCHITECTURAL SCALABILITY ROADMAP

Figure 5.1: NEXz Architectural Scalability Roadmap (Prototype to Institutional Deployment)

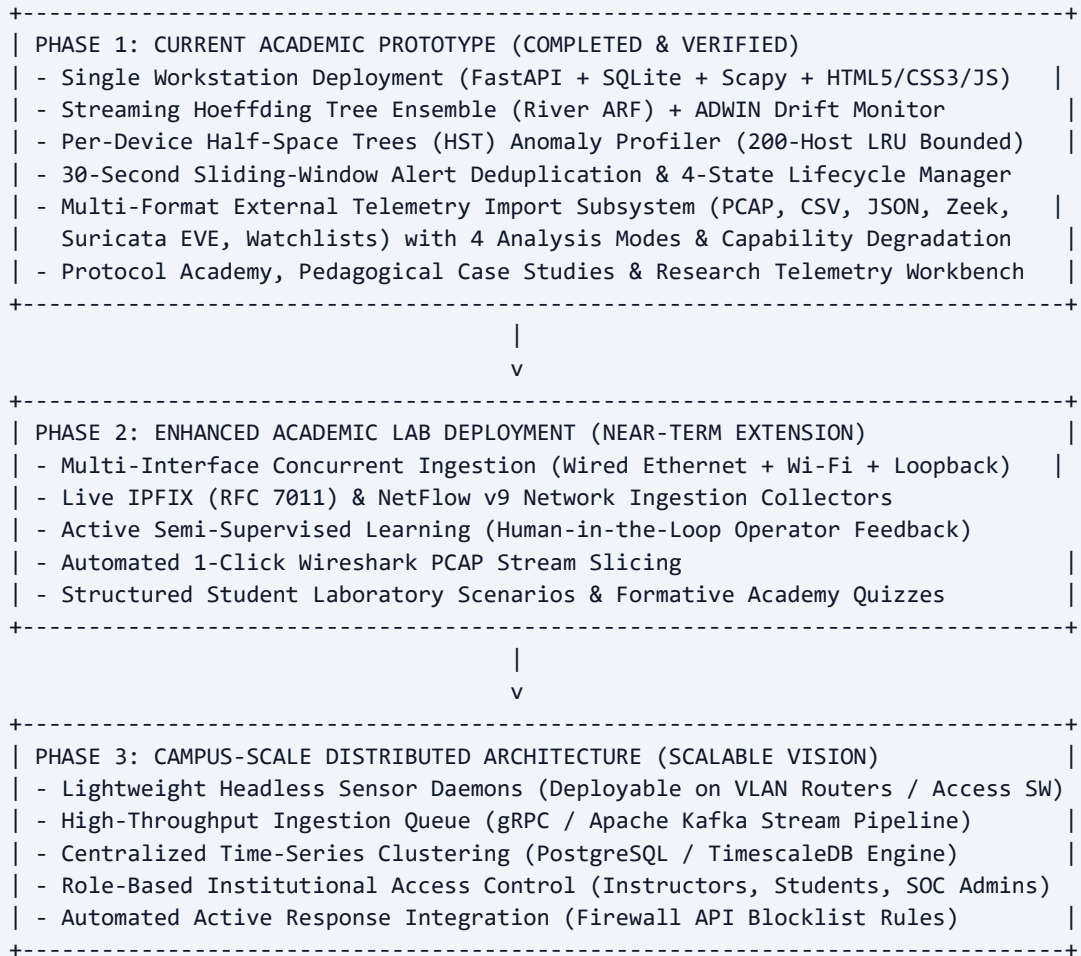


Figure 5.1 depicts the phased evolution of NEXz, illustrating how the current validated single-workstation prototype scales into an enterprise-grade distributed network security monitoring environment for institutional computer science departments.

REFERENCES

- Anderson, J. P. (1980). Computer security threat monitoring and surveillance (Technical Report). James P. Anderson Co., Fort Washington, PA.
- Bifet, A., & Gavaldà, R. (2007). Learning from time-changing data with adaptive windowing. In Proceedings of the 2007 SIAM International Conference on Data Mining (SDM) (pp. 443-448). Society for Industrial and Applied Mathematics. <https://doi.org/10.1137/1.9781611972771.42>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>
- Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2), 1153-1176. <https://doi.org/10.1109/COMST.2015.2494502>
- Combs, G. (2006). Wireshark: Network protocol analyzer. Software available from <https://www.wireshark.org>.
- Denning, D. E. (1987). An intrusion-detection model. *IEEE Transactions on Software Engineering*, SE-13(2), 222-232. <https://doi.org/10.1109/TSE.1987.232894>
- Domingos, P., & Hulten, G. (2000). Mining high-speed data streams. In Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining (pp. 71-80). <https://doi.org/10.1145/347090.347107>
- Gomes, H. M., Bifet, A., Read, J., Barddal, J. P., Enembreck, F., Pfahringer, B., Holmes, G., & Abdessalem, T. (2017). Adaptive random forests for evolving data stream classification. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 11(4), 1-24. <https://doi.org/10.1145/3127881>
- Kumar, P., Gupta, G. P., & Tripathi, R. (2024). Explainable AI for cyber threat intrusion detection: A comprehensive survey of methodologies, benchmark datasets, and open challenges. *Computers & Security*, 137, 103608. <https://doi.org/10.1016/j.cose.2023.103608>
- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2019). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346-2363. <https://doi.org/10.1109/TKDE.2018.2876857>
- Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS 2017)* (Vol. 30, pp. 4765-4774).
- Mohale, K., & Obagbuwa, I. C. (2025). Explainable artificial intelligence frameworks for network intrusion detection: Enhancing transparency and trust in automated security operations. *International Journal of Information Security*, 24(1), 45-62. <https://doi.org/10.1007/s10207-024-00892-x>
- Montiel, J., Halford, M., Mastelini, S. M., Bolmier, G., Sourty, R., Vaysse, R., Zouitine, A., Gomes, H. M., Read, J., Abdessalem, T., & Bifet, A. (2021). River: Machine learning for dynamic data in Python. *Journal of Machine Learning Research*, 22(110), 1-8.
- Oza, N. C., & Russell, S. (2001). Experimental comparisons of online and batch versions of bagging and boosting. In Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining (pp. 359-364).
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining (pp. 1135-1144). <https://doi.org/10.1145/2939672.2939778>
- Roesch, M. (1999). Snort: Lightweight intrusion detection for networks. In Proceedings of the 13th USENIX conference on System administration (LISA '99) (pp. 229-238).

Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In 2010 IEEE Symposium on Security and Privacy (pp. 305-316). IEEE.
<https://doi.org/10.1109/SP.2010.25>

Tan, S. C., Ting, K. M., & Liu, T. F. (2011). Fast anomaly detection for streaming data. In Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI-11) (pp. 1511-1516).

Widmer, G., & Kubat, M. (1996). Learning in the presence of concept drift and hidden contexts. *Machine Learning*, 23(1), 69-101. <https://doi.org/10.1007/BF00116900>